



TRABAJO FIN DE GRADO
GRADO EN INTELIGENCIA ARTIFICIAL

Visualizando el aprendizaje máquina: de la abstracción al entendimiento

Estudiante	Héctor Aldao Amoedo
Dirección:	Alejeandro Rodriguez Árias Samuel Magaz Romero

A Coruña, junio de 2026.

A Paulo González Ogando, mi profesor de matemáticas del instituto. Este es mi intento de explicar lo que he aprendido en esta carrera tan bien como él nos explicaba su asignatura.

Agradecimientos

A mi familia por el apollo infinito, a Bombardeen por la compañía y las risas, y a Las Damas por el tiempo y el cariño.

Resumen

Este trabajo presenta el diseño y desarrollo de una aplicación web educativa, implementada en Godot, para visualizar de forma interactiva el funcionamiento interno de dos modelos básicos de aprendizaje automático supervisado: los árboles de decisión y las redes neuronales multicapa. La herramienta está orientada a facilitar la comprensión de procesos que, estudiados únicamente mediante fórmulas, código o resultados finales, pueden resultar difíciles de interpretar de forma intuitiva. Para ello, la aplicación muestra la evolución de los algoritmos paso a paso y acompaña cada estado con explicaciones textuales y representaciones gráficas.

En el módulo de árboles de decisión se implementa un algoritmo basado en ID3 para atributos categóricos. Durante el entrenamiento, el usuario puede observar cómo se seleccionan los atributos mediante una métrica basada en la entropía y cómo se crean los nodos y las ramas. La fase de evaluación representa el recorrido de nuevos ejemplos por el árbol ya entrenado hasta obtener su clasificación. En el módulo de redes neuronales se implementa un perceptrón multicapa configurable, con selección de datos, definición de arquitectura, ejecución del *forward pass*, cálculo del error, retropropagación y actualización visual de pesos y sesgos. La aplicación incorpora ventanas informativas con fórmulas, gráficas de activación, carga de conjuntos de datos y persistencia de modelos. El resultado final se despliega como aplicación HTML5 mediante GitHub Pages, lo que permite acceder al recurso desde un navegador y utilizarlo como apoyo al aprendizaje de estos algoritmos.

Abstract

This work presents the design and development of an educational web application, implemented in Godot, for the interactive visualization of the internal behavior of two basic supervised machine learning models: decision trees and multilayer neural networks. The tool aims to support the understanding of processes that can be difficult to interpret intuitively when they are studied only through formulas, code or final results. To this end, the application displays the evolution of the algorithms step by step and links each state with textual explanations, graphical representations and animations integrated into the interface.

The decision tree module implements an ID3-based algorithm for categorical attributes. During training, the user can observe how attributes are selected through an entropy-based metric, how nodes and branches are created, and how final leaves are reached. The evaluation stage represents the path followed by new examples through the trained tree until a classification is obtained. The neural network module implements a configurable multilayer

perceptron, with dataset selection, architecture definition, *forward pass* execution, error calculation, backpropagation and visual updates of weights and biases. The application includes informative panels with formulas, activation graphs, dataset loading, validation mechanisms, model persistence and adaptation to different screen sizes. The final result is deployed as an HTML5 application through GitHub Pages, allowing the resource to be accessed from a browser and used as support for learning these algorithms.

Palabras clave:

- Aprendizaje automático
- Aprendizaje supervisado
- Visualización interactiva
- Godot
- Árboles de decisión
- Redes neuronales multicapa
- Aplicaciones web educativas

Keywords:

- Machine learning
- Supervised learning
- Interactive visualization
- Godot
- Decision trees
- Multilayer perceptrons
- Educational web applications

Índice general

1	Introducción	1
2	Estado del Arte	4
3	Herramientas y técnicas	8
3.1	Godot como motor de desarrollo	8
3.2	Control de versiones y despliegue	11
4	Metodología y planificación	13
4.1	Metodología de trabajo	13
4.2	Planificación por versiones	14
5	Marco teórico	17
5.1	Árboles de decisión e ID3	17
5.2	Redes neuronales multicapa	19
6	Desarrollo	22
6.1	Prototipo: versión 0.1	22
6.1.1	Primeras Escenas	22
6.1.2	Exportación web con GitHub Pages	24
6.2	Árbol de decisión: versión 0.2	24
6.2.1	Primera arquitectura del árbol	25
6.2.2	Reorganización de la Escena y primeros elementos explicativos	27
6.2.3	Animación de la construcción del árbol	29
6.2.4	Gráficas y casos explicativos en la ventana	31
6.2.5	Primer panel de selección de datos	32
6.2.6	Carga de archivos y navegación por pasos	34
6.2.7	Primera evaluación visual del árbol	35
6.3	Red neuronal: versión 0.3	37

6.3.1	Primera arquitectura visual de la red	37
6.3.2	Configuración de la arquitectura y representación lógica	39
6.3.3	Selección de datos y restricciones de la red	40
6.3.4	Algoritmo de entrenamiento incremental	42
6.3.5	Forward pass y visualización de datos	44
6.3.6	Cálculo del error y backward pass	45
6.3.7	Ventana informativa de la red neuronal	47
6.3.8	Inferencia con la red entrenada	48
6.4	Mejoras de funcionalidades e interfaz de usuario: versión 1.0	51
6.4.1	Entrada principal y navegación común	51
6.4.2	Adaptación a pantallas de distintos tamaños y orientaciones	53
6.4.3	Identidad visual y legibilidad de la interfaz	56
6.4.4	Renderizado de fórmulas matemáticas	58
6.4.5	Explicaciones de mayor nivel en la red neuronal	59
6.4.6	Gráficas de funciones de activación e inspección de neuronas y nodos	61
6.4.7	Selección de conjuntos de datos	62
6.4.8	Evaluación visual del árbol de decisión	63
6.4.9	Persistencia, carga y validación de modelos	64
6.4.10	Comprobación de resultados con bibliotecas externas	67
7	Conclusiones	70
8	Trabajo futuro	72
8.1	Ampliación del catálogo de modelos	72
8.2	Profundización en los modelos existentes	73
8.3	Integración con simulaciones interactivas	74
	Glosario	77
	Bibliografía	78

Índice de figuras

2.1	Sección interactiva de la página web de 3Blue1Brown.	6
2.2	Página web TensorFlow Playground.	6
3.1	Entorno de desarrollo de Godot.	9
3.2	Diagrama general de la arquitectura de Godot.	10
6.1	Versión preliminar del menú principal.	23
6.2	Primera Escena de prueba del árbol. Arriba a la derecha hay dos botones para guardar y cargar un árbol ya creado. Abajo a la izquierda aparece una estructura con forma de árbol, con nodos rojos y texto de ejemplo conectados por líneas negras. Sobre uno de los nodos se muestra el menú que permite añadirle un hijo o eliminarlo.	23
6.3	Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.	26
6.4	Separación de responsabilidades aplicada al árbol de decisión mediante el patrón Model-View-Controller.	26
6.5	Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.	27
6.6	Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.	28
6.7	Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.	29
6.8	Entrenamiento del árbol en el que se aprecian los nodos pendientes.	31

6.9	Ventana informativa en la que se dibuja la gráfica de la métrica basada en la entropía para cada atributo.	31
6.10	Menú que aparece tras seleccionar el modo algorítmico para escoger el conjunto de datos de entrenamiento.	33
6.11	Vista del árbol con el botón para retroceder al paso anterior.	34
6.12	Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.	36
6.13	Primera arquitectura visual de la red neuronal, formada por capas, neuronas y conexiones densas entre capas consecutivas.	38
6.14	A la izquierda el menú de creación de la red neuronal.	39
6.15	Selección de columnas objetivo para la red neuronal a partir de un conjunto de datos cargado desde un archivo CSV.	41
6.16	Red neuronal durante el entrenamiento, con nombres procedentes del conjunto de datos en las neuronas de entrada y salida.	42
6.17	Panel de entrenamiento de la red neuronal con controles para avanzar con distintos niveles de granularidad.	43
6.18	Red neuronal durante el entrenamiento, con la información del dato de entrada, su salida y su valor esperado.	45
6.19	Entrenamiento de la red en la que se ve en la ventana los pesos y bias de la neurona resaltada.	47
6.20	Entrenamiento de la red en la que se ve en la ventana una explicación en texto plano de la operación que se está calculando en la neurona resaltada.	48
6.21	Fórmula de derivada renderizada en la ventana.	49
6.22	Fórmula de derivada en texto plano en la ventana	49
6.23	Panel de selección de dataset de inferencia en el que se ha intentado seleccionar el dataset <i>EnemyAI</i> para una red entrenada con <i>Cultivos</i>	50
6.24	Mockup del menú principal realizado con Balsamiq antes de implementar la reorganización final de la entrada a la aplicación.	52
6.25	Versión final del menú principal, mostrando seleccionado el árbol de decisión.	52
6.26	Versión final del menú principal, mostrando seleccionada la red neuronal.	52
6.27	Menú de navegación que permite volver al menú principal y guardar o cargar estructuras de los algoritmos.	53
6.28	Mockups de las disposiciones horizontal y vertical de las vistas de algoritmo realizados con Balsamiq.	54
6.29	Vista del menu principal en un movil.	54
6.30	Creación de la red con el menú traslúcido situado sobre la visualización.	55
6.31	Creación de la red con el menú de creación situado al lado de la visualización.	55

6.32	Red neuronal con conexiones sin limite de grosor coloreadas mediante gradientes desde el cian hasta el verde para valores positivos y desde el amarillo hasta el rojo para valores negativos.	57
6.33	Red neuronal con conexiones con límite de grosor coloreadas en verde para valores positivos, en rojo para valores negativos, y en blanco los más cercanos a 0.	57
6.34	Red en medio de la animación de <i>forward pass</i>	58
6.35	Error en el que MathJax no se renderiza adecuadamente en el <i>canvas</i> de Godot	58
6.36	Texto y fórmulas de la propagación de la red por capas.	59
6.37	Texto y fórmulas de la retropropagación de la red por capas.	60
6.38	Texto y fórmulas al entrenar con dato en un paso.	60
6.39	Ejemplo de un nodo resaltado durante el entrenamiento de un árbol de decisión.	61
6.40	Etapa del <i>forward pass</i> en la que se ve en la ventana la función de activación dibujada junto con el punto que marca la salida de la función.	62
6.41	Diferentes estados de la carga de un dataset desde un csv.	62
6.42	Árbol de decisión durante el proceso de evaluación del conjunto de frutas. . .	64
6.43	Red neuronal entrenada dentro de la aplicación, con los pesos y el sesgo de la neurona de salida visibles en la interfaz.	66
6.44	Lectura externa del archivo ONNX exportado desde la aplicación, con los pesos y el sesgo de la red entrenada.	66
6.45	Red sintética exportada a ONNX desde un entorno externo, con pesos y sesgo conocidos.	66
6.46	Red sintética importada en la aplicación desde ONNX, con los pesos y el sesgo esperados visibles en la interfaz.	67
6.47	Primeros nodos de la estructura final de un árbol de decisión obtenido con scikit-learn a partir de un conjunto sintético.	68
6.48	Árbol de decisión generado por la aplicación con el mismo conjunto sintético usado en la referencia externa.	69

Índice de cuadros

4.1	Planificación del proyecto por versiones funcionales.	15
-----	---	----

Introducción

LA inteligencia artificial agrupa técnicas orientadas a construir sistemas capaces de resolver tareas que requieren predicción, razonamiento, reconocimiento de patrones o toma de decisiones. Dentro de este campo, el aprendizaje automático se centra en modelos que extraen regularidades a partir de datos para realizar predicciones sobre ejemplos nuevos. Una parte de estos modelos se basa en estructuras relativamente interpretables, como los árboles de decisión. El aprendizaje profundo (*deep learning*) emplea redes neuronales con varias capas para aprender representaciones progresivamente más complejas. Estos conceptos tienen una presencia creciente en ámbitos técnicos y cotidianos, lo que hace necesario contar con recursos que faciliten su comprensión más allá de su formulación matemática.

El estudio de estos algoritmos desde sus ecuaciones o desde una implementación convencional aporta rigor y permite conocer su funcionamiento formal. Al mismo tiempo, puede resultar difícil desarrollar una comprensión intuitiva del proceso completo cuando las operaciones se observan únicamente como código o fórmulas. En un árbol de decisión, la construcción del modelo depende de decisiones sucesivas sobre los atributos de entrada y sobre la incertidumbre de los datos. En una red neuronal, el entrenamiento implica el paso de información entre capas, el cálculo de errores, la propagación de gradientes y la actualización de pesos y sesgos. Estos procesos tienen una naturaleza dinámica, por lo que su comprensión mejora cuando pueden observarse paso a paso.

Las aplicaciones educativas interactivas ofrecen un medio adecuado para abordar esta dificultad. La visualización, la manipulación de ejemplos y la navegación por estados intermedios permiten relacionar conceptos abstractos con efectos visibles dentro de la interfaz. Este enfoque favorece el aprendizaje activo, ya que el estudiante puede avanzar, retroceder, cambiar datos o parámetros y comprobar cómo esas decisiones modifican el comportamiento del algoritmo. En este contexto, una herramienta interactiva puede actuar como un entorno de experimentación guiada en el que cada operación del modelo se asocia a una explicación textual, visual y numérica.

Este trabajo propone el desarrollo de una aplicación educativa web, implementada en Go-dot, para visualizar de forma interactiva el entrenamiento y la inferencia de dos modelos básicos de aprendizaje automático: un árbol de decisión y una red neuronal multicapa. La elección de estos modelos permite cubrir dos formas distintas de aprendizaje supervisado. El árbol de decisión representa el conocimiento mediante una estructura jerárquica de preguntas sobre los atributos de los datos. La red neuronal mediante operaciones que buscan transformar los datos de entrada en valores con significado. En ambos casos, la aplicación busca mostrar el proceso interno del algoritmo y acompañarlo con explicaciones integradas en la interfaz.

Los objetivos concretos del trabajo son los siguientes:

- Realizar un estudio conceptual de los algoritmos de inteligencia artificial seleccionados, entendiendo tanto el árbol de decisión como la red neuronal desde su funcionamiento a bajo nivel y desde sus parámetros de configuración.
- Desarrollar una aplicación educativa que permita visualizar de forma interactiva el paso a paso del entrenamiento y la inferencia de los dos algoritmos propuestos, acompañando cada proceso con texto informativo sobre las operaciones que se están llevando a cabo.
- Desplegar la aplicación en una web pública para facilitar su uso y su difusión como recurso de aprendizaje de los algoritmos propuestos.
- Adquirir, a nivel formativo, conocimientos sobre desarrollo de interfaces gráficas, despliegue de aplicaciones web y funcionamiento interno de los algoritmos estudiados.

El desarrollo de la aplicación se organiza alrededor de la relación entre algoritmo, representación visual e información explicativa. En el módulo del árbol de decisión se implementa una versión basada en ID3 con atributos categóricos. La aplicación permite observar la creación progresiva de nodos, la elección de atributos mediante una métrica basada en la entropía, la aparición de hojas y el recorrido posterior de ejemplos durante la evaluación. La ventana informativa acompaña cada paso con texto y gráficas para explicar por qué se toma cada decisión.

En el módulo de la red neuronal se implementa un perceptrón multicapa configurable. La interfaz permite definir una arquitectura adaptada al conjunto de datos, visualizar capas, neuronas y conexiones, ejecutar el *forward pass* paso a paso y mostrar cómo se calcula la salida de cada neurona. Durante el entrenamiento también se representa el cálculo del error, la retropropagación y la actualización de los parámetros. Después del entrenamiento, la aplicación permite evaluar nuevos ejemplos mediante la misma propagación hacia delante, con los pesos ya fijados.

La memoria refleja este proceso de trabajo. El Capítulo 2 revisa herramientas y recursos relacionados con la visualización educativa de modelos de aprendizaje automático. El Capí-

tulo 3 describe las tecnologías empleadas, con especial atención a Godot y su organización basada en Escenas, Nodos y Señales, y el despliegue mediante GitHub Pages. El Capítulo 5 recoge la base formal de los árboles de decisión y de las redes neuronales multicapa. El Capítulo 6 explica la evolución de la aplicación desde los primeros prototipos hasta la versión final, incluyendo la construcción de ambos módulos, la carga de datos, las animaciones, las explicaciones y las mejoras de interfaz. El Capítulo 7 resume los resultados obtenidos y valora el cumplimiento de los objetivos planteados. Y por último, en el Capítulo 8 se discuten posibles ampliaciones del proyecto.

El resultado final está publicado como aplicación web y puede consultarse desde un navegador en hectoraldao.github.io/visualizing_machine_learning. El código puede consultarse en el repositorio https://github.com/HectorAldao/visualizing_machine_learning. Esta publicación facilita que la herramienta pueda utilizarse como recurso de apoyo para estudiar el funcionamiento interno de los modelos implementados, experimentar con datos y configuraciones, y observar de forma guiada cómo un algoritmo de aprendizaje automático pasa de los datos de entrada a una predicción.

Estado del Arte

LA inteligencia artificial y el aprendizaje automático se apoyan en algoritmos cuyo funcionamiento interno no siempre resulta fácil de visualizar. Conceptos como el cálculo de una frontera de decisión o la actualización iterativa de los parámetros de un modelo pueden resultar abstractos cuando se estudian únicamente desde una formulación teórica. Esta dificultad se acentúa en el aprendizaje profundo, donde intervienen elementos como funciones de activación, pesos, sesgos, retropropagación o gradientes. Por ello, se han desarrollado recursos educativos orientados a hacer más comprensible el comportamiento de estos modelos mediante explicaciones visuales y simulaciones interactivas. En el contexto de este trabajo, resultan especialmente relevantes las herramientas que permiten comprender el proceso de aprendizaje de los modelos, la relación entre sus componentes internos y la forma en la que una decisión algorítmica se transforma en un resultado observable.

Uno de los recursos divulgativos más conocidos para introducir las redes neuronales es la serie de vídeos *Neural networks* [1]. Esta fue publicada en el canal de YouTube *3Blue1Brown*. Este proyecto cuenta también con una página web [2], cuya principal aportación pedagógica es la posibilidad de interactuar con una red neuronal entrenada con el conjunto de datos MNIST [3]. La herramienta permite observar en tiempo real cómo la red reacciona a los cambios en el dato de entrada, representado mediante una superficie de dibujo en la que cada píxel se corresponde con una neurona de la capa de entrada, como se aprecia en la Figura 2.1.

Este tipo de recurso resulta valioso para desarrollar intuición, ya que reduce la barrera de entrada matemática y proporciona una primera aproximación visual al funcionamiento de una red. También es adecuado para motivar y contextualizar el problema gracias a una explicación apoyada en recursos visuales claros. Su formato, en cambio, conserva un carácter fundamentalmente expositivo, ya que en la página web, solo dos de las diez lecciones incorporan un componente interactivo, y dicha interacción se orienta siempre a la relación entre la entrada y la salida de una red ya entrenada. El estudiante trabaja con una explicación limitada al conjunto MNIST y sin capacidad de experimentar de forma directa con la arquitectura del

modelo. Esta restricción deja abierta la necesidad de recursos más activos, en los que sea posible analizar el efecto de cada decisión de diseño sobre el comportamiento de una red neuronal.

Frente al formato audiovisual, existen herramientas que permiten manipular redes neuronales en tiempo real. Un ejemplo es *A Neural Network Playground* [4], una aplicación que permite configurar una red neuronal sencilla desde el navegador, seleccionar conjuntos de datos sintéticos, modificar hiperparámetros y observar cómo evoluciona la frontera de decisión durante el entrenamiento. En la Figura 2.2 se puede visualizar la página web.

A diferencia de los recursos puramente expositivos, la relevancia de TensorFlow Playground reside en que conecta elementos habitualmente tratados de forma abstracta con consecuencias observables. Modificar directamente la arquitectura del modelo (número de capas y neuronas) o sus hiperparámetros (tasa de aprendizaje y regularización) altera de manera inmediata el proceso de ajuste. Este enfoque interactivo facilita que el estudiante relacione conceptos críticos como el sobreajuste, la generalización o la convergencia con resultados palpables. La herramienta se centra principalmente en la frontera de decisión y en la pérdida, sin profundizar de forma detallada en operaciones internas como el cálculo de gradientes o la propagación de errores capa a capa.

Esta misma orientación hacia la experimentación aparece en herramientas centradas en arquitecturas más específicas. *GAN Lab* [5] permite experimentar en el navegador con redes generativas adversarias. Este tipo de modelo introduce una dinámica de aprendizaje distinta a la de una red supervisada clásica, ya que enfrenta dos redes: un generador, que intenta producir muestras plausibles, y un discriminador, que intenta distinguir entre datos reales y generados. La aportación principal de *GAN Lab* es representar visualmente una interacción compleja como la de un par de redes adversarias. La herramienta permite observar cómo cambian las distribuciones generadas durante el entrenamiento y cómo la competencia entre generador y discriminador afecta al resultado. Desde el punto de vista educativo, esto resulta útil para explicar que existen modelos de aprendizaje profundo con objetivos y esquemas de aprendizaje diferentes.

La limitación principal de *GAN Lab*, junto con la falta de detalle sobre ciertas operaciones internas, es que presupone una base de conocimiento mayor que la requerida para una introducción a las redes neuronales. Para comprender correctamente una GAN es necesario haber asimilado previamente ideas como función de pérdida, optimización, representación interna o entrenamiento iterativo. En consecuencia, *GAN Lab* es una herramienta valiosa para una fase posterior del aprendizaje, aunque no sustituye a una aplicación centrada en los fundamentos de una red neuronal básica.

En el ámbito de los algoritmos clásicos de aprendizaje automático, también existen propuestas para ilustrar el funcionamiento de los árboles de decisión. La herramienta web interactiva sobre árboles de decisión de Interactive Machine Learning [6] permite al usuario

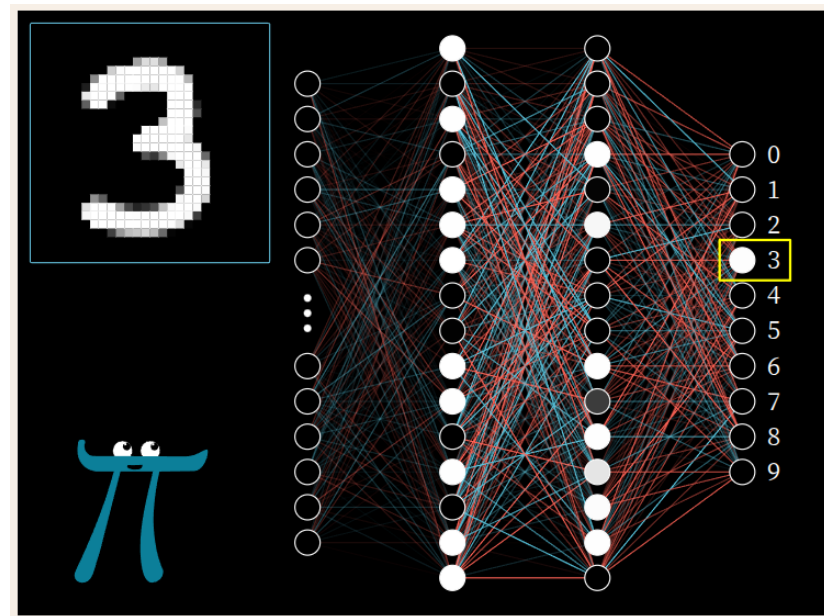


Figura 2.1: Sección interactiva de la página web de 3Blue1Brown.

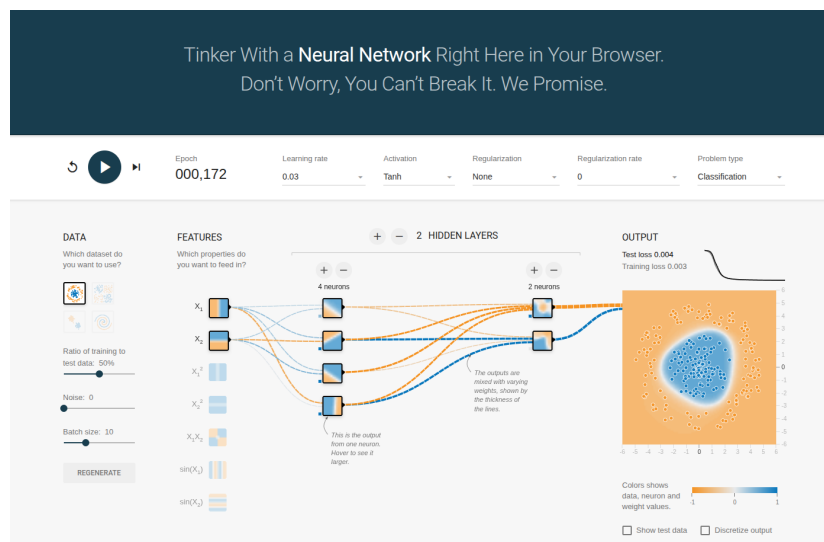


Figura 2.2: Página web TensorFlow Playground.

construir un árbol y observar el proceso de división de un conjunto de datos en tiempo real. Esta propuesta resulta interesante porque acerca al estudiante a la estructura jerárquica del modelo y al proceso de partición sucesiva de los datos. Su experiencia de uso, no obstante, es limitada y las explicaciones que acompañan a la simulación tienen poco desarrollo.

Aunque muestra los valores de distintas métricas, no acompaña dichos valores con una interpretación suficiente de su significado ni de su papel dentro del algoritmo. Esto reduce su utilidad como recurso formativo autónomo, especialmente para estudiantes que se aproximan por primera vez a los árboles de decisión.

En comparación con las redes neuronales, la disponibilidad de herramientas interactivas centradas en árboles de decisión es más reducida. La revisión realizada muestra una presencia mucho mayor de simuladores y visualizaciones dedicados a redes neuronales, tanto en recursos divulgativos como en herramientas web orientadas a la experimentación. Esta diferencia es especialmente visible cuando se buscan recursos que combinen visualización del modelo, explicación del criterio de partición y evaluación paso a paso.

Desde la perspectiva de este proyecto, la principal oportunidad se encuentra en combinar las fortalezas de estos recursos. Una herramienta educativa eficaz debería mantener la claridad visual de los recursos divulgativos y su atención a la base matemática, incorporar la manipulación directa de los simuladores y ofrecer suficiente rigor técnico para que lo presentado al usuario se corresponda con el funcionamiento real del algoritmo. También debería evitar que la interacción se limite a modificar hiperparámetros globales. Para entender una red neuronal resulta especialmente útil poder observar el flujo de datos, la contribución de los pesos y el efecto del entrenamiento sobre cada componente del modelo. En el caso de los árboles de decisión, la visualización debería mostrar cómo se selecciona cada atributo, cómo se divide el conjunto de datos y cómo se alcanza una clasificación final a partir de las características de entrada. En ambos modelos, esta visualización debe ir acompañada de explicaciones textuales que interpreten las métricas, los cálculos y las decisiones del algoritmo, de forma que la simulación no dependa únicamente de la observación visual.

Herramientas y técnicas

EL desarrollo de este trabajo se apoya en un conjunto de herramientas orientadas a construir una aplicación educativa, interactiva y ejecutable desde el navegador. La elección tecnológica del entorno de desarrollo tiene en cuenta tanto criterios de implementación como la naturaleza del problema abordado, siendo este explicar modelos de aprendizaje automático mediante visualizaciones dinámicas y ejemplos manipulables por el usuario. En consecuencia, se priorizan tecnologías que facilitan la construcción de interfaces interactivas, la validación de los algoritmos implementados y la publicación del resultado final en un entorno accesible. Godot permite exportar un mismo proyecto a distintas plataformas, como escritorio, móvil o web. En este caso, la versión web se escogió por su accesibilidad al permitir que el usuario puede abrir la aplicación desde un navegador, sin instalar software ni preparar un entorno de ejecución específico.

3.1 Godot como motor de desarrollo

La herramienta principal a emplear es Godot, un motor de desarrollo libre y multiplataforma orientado a la creación de aplicaciones interactivas [7]. Sus características resultan adecuadas para una aplicación educativa como la desarrollada en este trabajo debido a que el proyecto requiere presentar información visual, responder a acciones del usuario, animar procesos paso a paso y mantener una estructura clara entre los distintos elementos de la interfaz. Estas necesidades encajan de forma natural con el modelo de Escenas, Nodos y Señales que propone Godot [8].

Los Nodos son el elemento básico y más importante con el que se trabaja en el motor. Todo elemento y subelemento que vaya a estar presente en tiempo de ejecución en el programa es un Nodo. Y cada Nodo cuenta, principalmente, con una serie de atributos, métodos y Señales. Igual que en la programación orientada a objetos, los atributos son variables o constantes y los métodos funciones. Las Señales son disparadores de funciones. Cuando son emitidas todas las

funciones a las que están conectadas son llamadas. Son una forma de conectar eventos entre diferentes **Nodos**. Son el núcleo de la programación dirigida por eventos, el paradigma de programación que promueve el motor.

Los atributos, métodos y Señales de cada **Nodo** vienen definidos por su clase. Godot dispone de un gran árbol de clases predefinidas que heredan atributos y métodos de sus clases padre. Las tres clases básicas son: el *Node* una clase vacía, la raíz del resto de **Nodos**, que solo cuenta con los métodos mínimos para que funcione en el motor; el *Node2D* característico por contar con atributos referentes a la posición en un espacio bidimensional; el *Node3D* similar al anterior pero en un espacio tridimensional; y el *Control* pensado para los elementos de interfaz de usuario. Por encima de estas, el desarrollador siempre puede definir sus propias clases ampliando las ya existentes.

En este proyecto, los **Nodos Control** son los más importantes. Esto se debe a que como ningún elemento a representar necesita habitar un espacio ni estar sujeto a físicas para poder ser representado, no hace falta que sean *Node2D* ni *Node3D*, al tiempo que los *Control* ofrecen grandes ventajas a la hora de usarlos para interactuar con el usuario.

Los **Nodos** por sí solos son clases abstractas que no existen en el programa ejecutable final por sí mismos. Para instanciarlos están las **Escenas**. Estas son la materialización concreta del proyecto, los archivos que son guardados en disco y con los que se interactúa en el editor como se puede ver en la Figura 3.1. Están formadas, como mínimo, por un **Nodo** raíz. De este **Nodo** raíz pueden colgar **Nodos** hijo, que a su vez pueden contar con sus propios **Nodos** hijo, y así sucesivamente.

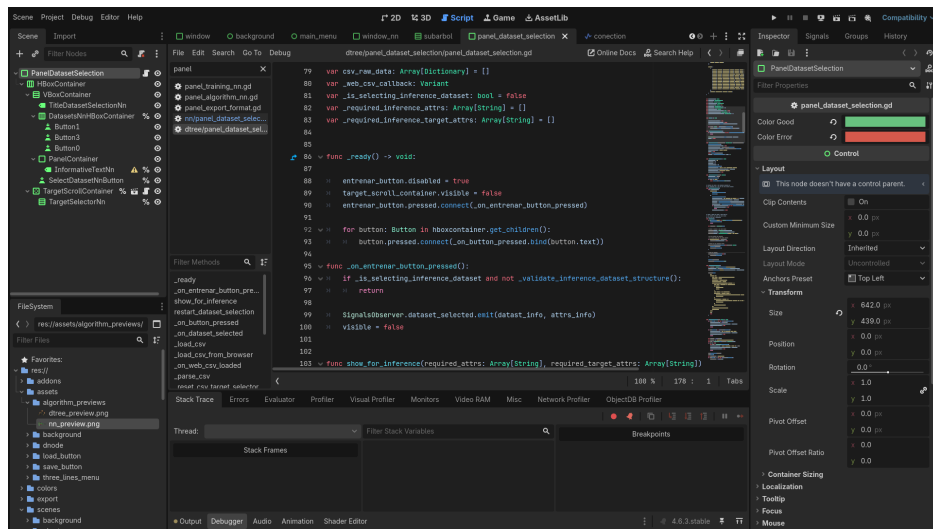


Figura 3.1: Entorno de desarrollo de Godot.

Finalmente, las **Escenas**, como en última instancia son árboles de **Nodos**, pueden ser instanciadas dentro de otras **Escenas**. Esto habilita una modularidad total, puesto que toda es-

estructura de **Nodos** que sea susceptible de ser reutilizada, puede guardarse como una **Escena** para no tener que ser recreada.

Además, los **Nodos** instanciados pueden tener asociado un **Script**. Estos son código desde el que se puede interactuar, principalmente, con los atributos y métodos del **Nodo** al que están asociados, así como con sus hijos. Pero no están limitados a estos elementos. Siempre se puede acceder al **Nodo** raíz y, desde este, modificar cualquier propiedad global o acceder a cualquier **Nodo** en la **Escena**. Y los **Scripts** pueden ser independientes de cualquier **Escena** mediante la creación de *Autoloads* en el proyecto.

La Figura 3.2 muestra de forma esquemática esta organización y sitúa las **Escenas**, los **Nodos**, los **Scripts** y los módulos dentro de la arquitectura general del motor.

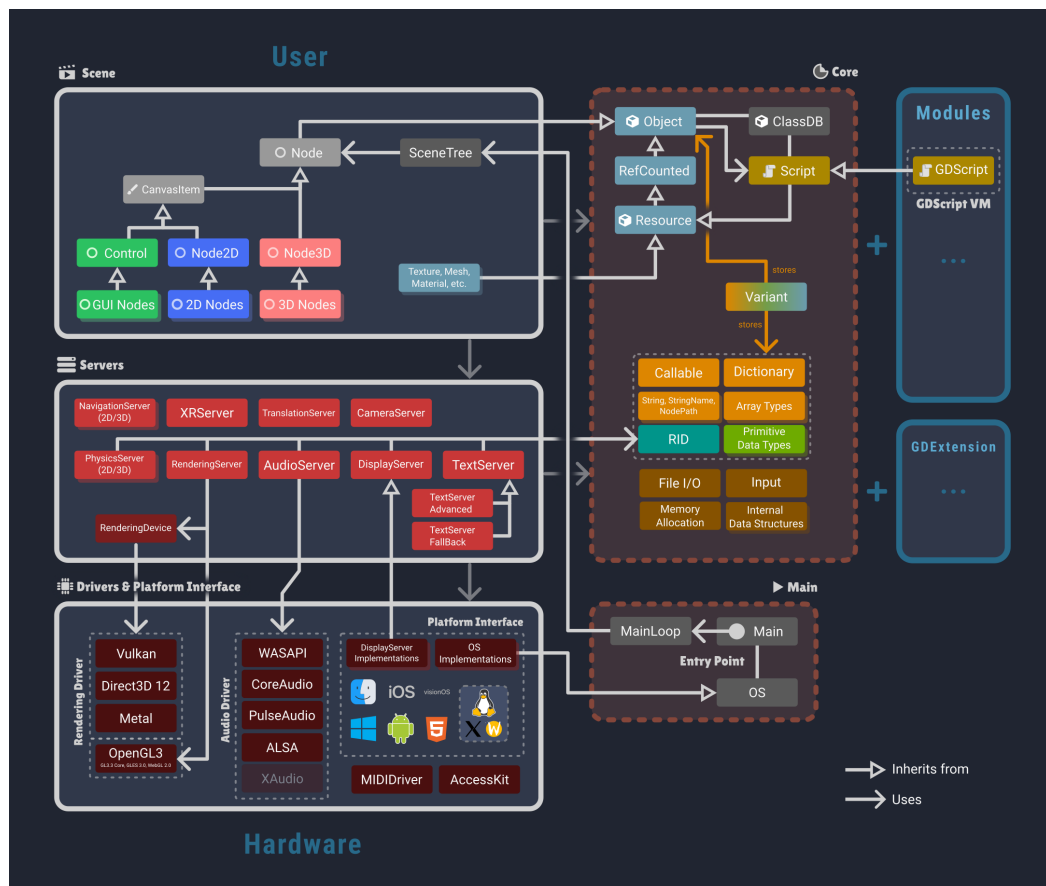


Figura 3.2: Diagrama general de la arquitectura de Godot.

La organización en **Escenas** permite separar componentes de la aplicación con responsabilidades distintas, como las vistas de entrenamiento, los paneles explicativos, las representaciones gráficas y los controles de interacción. Este enfoque favorece una estructura modular en la que cada parte de la interfaz puede desarrollarse, probarse y reutilizarse con menor acoplamiento.

El sistema de Señales facilita la comunicación entre elementos de la aplicación sin imponer dependencias directas innecesarias, lo que resulta útil en un entorno en el que las acciones del usuario deben provocar cambios visuales y actualizaciones del estado interno de los modelos.

Otro motivo importante para escoger Godot fue su sistema de exportación multiplataforma. El motor permite generar versiones para diferentes entornos a partir del mismo proyecto, incluida una versión HTML5 de la aplicación [9]. La exportación web se adoptó por la forma en la que se quería distribuir la herramienta, puesto que una aplicación educativa debe ser sencilla de abrir, compartir y usar. Publicarla como aplicación web evita exigir al usuario una instalación local y facilita el acceso desde un navegador. Esta característica es especialmente relevante en un proyecto con finalidad educativa, ya que reduce la fricción de uso y permite distribuir la herramienta como una aplicación web estática.

3.2 Control de versiones y despliegue

Para la gestión del código fuente se utilizó Git, un sistema de control de versiones distribuido ampliamente utilizado [10]. Su uso permitió mantener un registro de los cambios y trabajar de forma incremental. Esto resultó útil durante el desarrollo, ya que la aplicación fue creciendo mediante pruebas sucesivas: primero se validó la interacción básica, después se incorporaron los algoritmos y más adelante se añadieron explicaciones y mejoras de interfaz. Disponer de un historial de cambios permitió avanzar con mayor seguridad y conservar puntos estables del proyecto mientras se realizaban modificaciones.

El repositorio se alojó en GitHub, lo que proporcionó almacenamiento remoto y una copia accesible del proyecto. Además, se utilizó GitHub Pages para publicar la versión web de la aplicación [11]. Este servicio permite desplegar sitios estáticos directamente desde un repositorio, por lo que encaja con la exportación web generada por Godot. La aplicación exportada se publica como un conjunto de archivos preparados para ejecutarse en el navegador, distinto del proyecto editable de Godot.

Esta fase tiene importancia porque marca la diferencia entre una aplicación que funciona dentro del entorno de desarrollo y una herramienta que puede utilizar cualquier persona desde un enlace. En el editor, el proyecto se ejecuta en un contexto controlado, con acceso directo a las Escenas, Scripts y recursos locales. En el navegador, la aplicación depende de los archivos exportados, de las restricciones propias de la plataforma web y de la forma en la que el servicio de publicación sirve esos archivos. Por este motivo, el despliegue se trató como una parte más del desarrollo y no únicamente como el último paso administrativo.

La combinación de Godot y GitHub Pages permitió que el resultado final pudiese consultarse desde el navegador sin infraestructura de servidor adicional. Esto encaja con el objetivo de accesibilidad planteado al escoger la exportación web, puesto que el usuario solo necesi-

ta abrir una dirección. Y al mismo tiempo el proyecto mantiene un proceso de publicación suficientemente ágil para ser actualizado durante el desarrollo.

Metodología y planificación

EL desarrollo del proyecto se organizó mediante una metodología incremental basada en Scrum. Esta elección resulta adecuada para un trabajo en el que conviven investigación técnica, implementación de algoritmos, diseño de interfaz, validación visual y despliegue web, ya que permite avanzar mediante incrementos funcionales y revisar de forma periódica el avance logrado. La definición completa de la aplicación dependía de comprobar previamente la viabilidad de varios aspectos, como la visualización, la exportación a navegador, la carga de datos y la explicación paso a paso de cada algoritmo. Por ello, la planificación se planteó como una sucesión de versiones, cada una asociada a un objetivo concreto y a un resultado evaluable.

4.1 Metodología de trabajo

Scrum es un marco de trabajo ágil orientado al desarrollo de productos complejos mediante ciclos de duración acotada, llamados *Sprints*, en los que se selecciona un conjunto de tareas procedentes del *Product Backlog* y se obtiene un incremento revisable del producto [12]. El marco se apoya en la transparencia del estado del trabajo, la inspección frecuente de los resultados y la adaptación de la planificación a partir de lo aprendido durante el desarrollo. Estos principios encajan con un proyecto educativo interactivo, donde cada avance debe evaluarse tanto desde el punto de vista técnico como desde su capacidad para explicar correctamente el funcionamiento de los modelos.

En este trabajo se aplicó Scrum de forma adaptada al contexto académico de un trabajo de fin de grado. El estudiante asumió las tareas de análisis, implementación, prueba y documentación, mientras que los tutores participaron en el seguimiento, la revisión técnica y la orientación del alcance. La pila de producto agrupó las funcionalidades, validaciones y mejoras pendientes, desde prototipos de interfaz y adaptación web, a implementación de los algoritmos. Todas las tareas se priorizaron según su dependencia con otras partes del sistema.

La planificación se estructuró en Sprints asociados a versiones funcionales del proyecto. Cada Sprint perseguía un incremento concreto de la aplicación, de forma que al finalizarlo existiese una versión utilizable y revisable. Los Sprints fueron: el prototipo inicial, el módulo del árbol de decisión, el módulo de la red neuronal y la fase de mejoras funcionales e interfaz. Las reuniones se realizaban semanalmente con los tutores y funcionaban como puntos de inspección y adaptación dentro de la versión activa. En ellas se revisaba el trabajo completado desde la reunión anterior, se analizaban las dificultades encontradas, se validaban las decisiones tomadas y se definían las tareas prioritarias para continuar.

Estas reuniones cumplían una función cercana a la *Sprint Review*, porque permitían inspeccionar el incremento parcial y contrastarlo con los objetivos del trabajo. También incorporaban elementos de planificación, ya que al final de cada sesión se concretaban las tareas que debían abordarse durante la siguiente semana dentro del Sprint en curso. Cuando una decisión técnica generaba problemas de mantenimiento, legibilidad o interacción, la reunión servía también como espacio de retrospectiva, revisando la forma de trabajo y ajustando el enfoque para las siguientes tareas.

La revisión semanal con los tutores permitió detectar problemas antes de que se acumulasen en fases posteriores. De esta forma, la metodología seguida mantuvo un equilibrio entre planificación inicial y adaptación continua, conservando siempre una versión funcional como referencia del avance del proyecto.

4.2 Planificación por versiones

La división por versiones permitió mantener una relación directa entre planificación y desarrollo. Cada versión incorporó un conjunto coherente de funcionalidades y dejó preparada la base para la siguiente. La Tabla 4.1 resume esta organización.

Sprint	Objetivo principal	Incremento alcanzado
Prototipo, versión 0.1	Validar Godot como tecnología para una aplicación interactiva exportable a web.	Menú inicial, <i>Escena</i> de prueba con una estructura jerárquica editable, creación y eliminación de <i>nodos</i> , conexiones visuales y primera exportación mediante GitHub Pages.
Árbol de decisión, versión 0.2	Implementar el primer algoritmo y representar su entrenamiento e inferencia de forma visual.	Árbol basado en ID3, construcción paso a paso, <i>nodos</i> internos y hojas, ramas etiquetadas, ventana informativa, gráficas de entropía, carga de datos, retroceso por pasos y evaluación visual.
Red neuronal, versión 0.3	Incorporar una red multicapa configurable y mostrar su entrenamiento incremental.	Arquitectura visual por capas, selección de datos, restricciones de entrada y salida, forward pass, cálculo del error, backward pass, actualización visible de pesos, ventana informativa e inferencia.
Mejoras funcionales e interfaz, versión ¿?	Consolidar la aplicación como herramienta web coherente y reutilizable.	Navegación común, adaptación a distintos tamaños de pantalla, identidad visual, renderizado de fórmulas, validaciones de datos, mejoras en evaluación, persistencia de modelos y comprobaciones con bibliotecas externas.

Cuadro 4.1: Planificación del proyecto por versiones funcionales.

El primer Sprint se centró en reducir la incertidumbre técnica inicial. Antes de implementar los modelos de aprendizaje automático, era necesario comprobar que Godot permitía construir una interfaz dinámica, actualizar estructuras visuales durante la ejecución y publicar el resultado en un navegador. El prototipo de la versión 0.1 sirvió como prueba de concepto y asentó las bases para la organización y el despliegue web que se reutilizaría en las siguientes fases.

El segundo Sprint tuvo como objetivo convertir esa base visual en un módulo funcional de árbol de decisión. La planificación de esta versión se organizó alrededor de la construcción incremental del algoritmo. Primero, la representación lógica y visual del árbol. Después el avance paso a paso junto a la explicación de las decisiones mediante texto y métricas. Y por último la carga de datos y la evaluación de ejemplos nuevos. Esta secuencia permitió validar de forma progresiva la correspondencia entre el estado interno del algoritmo y la *escena* mostrada al usuario.

El tercer Sprint incorporó el segundo modelo previsto, la red neuronal multicapa. Su planificación partió de la experiencia obtenida con el árbol, manteniendo la separación entre lógica del algoritmo, representación visual y ventana informativa. En esta versión el trabajo se orientó a construir una arquitectura configurable, adaptarla al conjunto de datos, ejecutar el forward pass y el backward pass de forma incremental y mostrar las actualizaciones de pesos y sesgos como parte del entrenamiento.

El cuarto y último Sprint agrupó las mejoras necesarias para transformar los módulos ya funcionales en una aplicación más completa. En esta fase se revisó la navegación general, se adaptaron las vistas a pantallas de distinto tamaño, se mejoró la legibilidad visual, se integró el renderizado de fórmulas matemáticas, se ampliaron las validaciones de datos y se añadieron funciones de persistencia. También se comprobaron los resultados de los algoritmos mediante herramientas externas, reforzando la confianza en la correspondencia entre los cálculos internos y la información presentada por la interfaz.

Marco teórico

Este capítulo recoge la base formal de los dos modelos implementados en la aplicación. En primer lugar se presentan los árboles de decisión construidos mediante ID3 con atributos categóricos y entropía. Después se describe el funcionamiento de una red neuronal multicapa sencilla, formada por capas densas y entrenada mediante retropropagación.

5.1 Árboles de decisión e ID3

Los árboles de decisión son modelos supervisados que organizan una clasificación como una secuencia de preguntas sobre los atributos de entrada. Cada nodo interno representa una pregunta, cada rama representa una posible respuesta y cada hoja contiene la etiqueta predicha. El algoritmo ID3 fue presentado por Quinlan como un procedimiento para inducir árboles de decisión a partir de ejemplos etiquetados [13]. La versión considerada en este trabajo usa atributos categóricos y selecciona las divisiones mediante el cálculo de la ganancia de información basada en la entropía.

Sea D un conjunto de ejemplos de entrenamiento y sea C el conjunto de clases posibles. Para una clase $c \in C$, se define su probabilidad como:

$$p(c) = \frac{|\{x \in D : y(x) = c\}|}{|D|},$$

donde $y(x)$ es la etiqueta del ejemplo x . La entropía del conjunto D mide la incertidumbre sobre la clase de un ejemplo tomado de ese conjunto:

$$H(D) = - \sum_{c \in C} p(c) \log_2 p(c).$$

Cuando $p(c) = 0$, el término $p(c) \log_2 p(c)$ se toma como 0. Si todos los ejemplos pertenecen a la misma clase, la entropía vale 0. Si las clases aparecen repartidas de forma más equilibrada,

la entropía aumenta.

Para evaluar un atributo categórico A , ID3 separa D en subconjuntos según los valores posibles de ese atributo. Si $\mathcal{V}(A)$ es el conjunto de valores de A , se define

$$D_v = \{x \in D : A(x) = v\}, \quad v \in \mathcal{V}(A).$$

La entropía esperada tras dividir por A es la media ponderada de las entropías de esos subconjuntos:

$$H(D \mid A) = \sum_{v \in \mathcal{V}(A)} \frac{|D_v|}{|D|} H(D_v).$$

La ganancia de información indica cuánto se reduce la incertidumbre al usar ese atributo:

$$IG(D, A) = H(D) - H(D \mid A).$$

ID3 escoge el atributo con mayor ganancia de información:

$$A^* = \arg \max_{A \in \mathcal{A}} IG(D, A),$$

donde $\mathcal{A}$ es el conjunto de atributos disponibles en ese nodo.

El algoritmo se aplica de forma recursiva. En cada nodo se comprueba primero si todos los ejemplos tienen la misma clase. En ese caso se crea una hoja con esa etiqueta. Si todavía hay varias clases y quedan atributos disponibles, se escoge el atributo con mayor ganancia de información y se crea una rama para cada valor observado. Después, el mismo proceso se repite sobre cada subconjunto D_v , retirando el atributo usado para que no vuelva a seleccionarse en las ramas inferiores. Si se agotan los atributos antes de conseguir subconjuntos puros, se crea una hoja con la clase mayoritaria del conjunto que llega a ese nodo:

$$\hat{c} = \arg \max_{c \in C} |\{x \in D : y(x) = c\}|.$$

La formulación anterior corresponde a la versión básica de ID3 usada en la aplicación. Existen extensiones y variantes que modifican varios puntos del proceso, como tratar atributos continuos mediante umbrales, utilizar otras métricas como el índice de Gini o la razón de ganancia, aplicar técnicas de poda para reducir sobreajuste o permitir árboles con restricciones distintas sobre su profundidad. En este trabajo se fija el caso categórico con entropía porque permite mostrar de forma directa cómo una métrica numérica justifica cada partición del conjunto de datos.

5.2 Redes neuronales multicapa

Las redes neuronales artificiales tienen su origen en modelos como el perceptrón de Rosenblatt [14]. La versión utilizada en este trabajo corresponde a un perceptrón multicapa, es decir, una red formada por una capa de entrada, cero o más capas ocultas y una capa de salida. El entrenamiento de redes multicapa mediante retropropagación fue popularizado por Rumelhart, Hinton y Williams [15], y constituye la base del procedimiento implementado en la aplicación.

Una red neuronal densa se organiza en capas numeradas desde 0 hasta L . La capa 0 es la capa de entrada y contiene el vector de atributos:

$$a^{(0)} = x.$$

Para cada capa posterior $l \in \{1, \dots, L\}$, el modelo dispone de una matriz de pesos $W^{(l)}$, un vector de sesgos $b^{(l)}$ y una función de activación f_l . El cálculo de la capa se divide en dos pasos. Primero se obtiene la combinación lineal de la entrada de la capa por sus pesos más el sesgo:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}.$$

Y después se aplica la función de activación:

$$a^{(l)} = f_l(z^{(l)}).$$

El vector $a^{(l)}$ pasa a ser la entrada de la capa siguiente. La salida final de la red es $a^{(L)}$.

Las funciones de activación introducen transformaciones no lineales o adaptan la salida al tipo de problema. Entre las funciones habituales se encuentran la identidad, usada a menudo en regresión,

$$f(z) = z,$$

la sigmoide,

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

y ReLU,

$$\text{ReLU}(z) = \max(0, z).$$

En clasificación multiclase se usa con frecuencia softmax, que transforma un vector de valores en una distribución de probabilidades:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_k e^{z_k}}.$$

Softmax se aplica al vector completo de la capa, ya que cada salida depende de todos los componentes del vector z .

Para entrenar la red hace falta definir una función de pérdida L , que mide la diferencia entre la salida calculada $a^{(L)}$ y el objetivo esperado y . En regresión puede emplearse el error cuadrático medio, expresado para una muestra como

$$L = \frac{1}{2} \|a^{(L)} - y\|^2.$$

En clasificación con softmax suele usarse entropía cruzada:

$$L = - \sum_i y_i \log a_i^{(L)}.$$

El entrenamiento busca ajustar los pesos y sesgos para reducir esta pérdida.

La retropropagación calcula cómo afecta cada parámetro a la pérdida mediante la regla de la cadena. Para ello se define un término de error o delta en cada capa:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}.$$

En la capa de salida, cuando se combina softmax con entropía cruzada, aparece una simplificación muy usada:

$$\delta^{(L)} = a^{(L)} - y.$$

Para otras combinaciones de pérdida y activación, el delta de salida se obtiene multiplicando el gradiente de la pérdida respecto a la activación por la derivada local de la activación:

$$\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot f'_L(z^{(L)}),$$

donde $\odot$ indica producto elemento a elemento.

En las capas ocultas, el error se obtiene a partir de los deltas de la capa siguiente:

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot f'_l(z^{(l)}).$$

Esta expresión resume la contribución de la capa l al error posterior y la ajusta según la pendiente local de su función de activación.

Una vez conocidos los deltas, los gradientes de los parámetros de cada capa quedan definidos como

$$\begin{aligned} \frac{\partial L}{\partial W^{(l)}} &= \delta^{(l)} \left(a^{(l-1)}\right)^T, \\ \frac{\partial L}{\partial b^{(l)}} &= \delta^{(l)}. \end{aligned}$$

Con descenso de gradiente estocástico, los parámetros se actualizan tras cada muestra con una tasa de aprendizaje η :

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}},$$
$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}.$$

Este ciclo se repite sobre todas las muestras de entrenamiento. Cada muestra produce una propagación hacia delante, una pérdida, una retropropagación de deltas y una actualización de parámetros.

Durante la inferencia se utiliza únicamente la propagación hacia delante. Los pesos y sesgos permanecen fijos, la entrada atraviesa las capas mediante las mismas ecuaciones y la salida final se interpreta como predicción del modelo.

Capítulo 6

Desarrollo

En este capítulo se describe el desarrollo de la aplicación a través de las versiones principales del proyecto. La exposición sigue el orden en el que se incorporaron las funcionalidades, desde la validación inicial de Godot como tecnología de desarrollo y despliegue web hasta la implementación del árbol de decisión, la red neuronal y los ajustes finales de la interfaz.

De ahora en adelante cuando se hace referencia a elementos propios de Godot se mantiene la mayúscula inicial, como en [Escena](#) y [Nodo](#), para diferenciarlos de los usos generales de [escena](#) y [nodo](#).

6.1 Prototipo: versión 0.1

El primer prototipo tiene como objetivo validar que Godot puede servir como base para crear una aplicación interactiva exportable a web. Antes de avanzar hacia la implementación de los algoritmos, se comprobó que el motor permitía construir una interfaz básica, representar una estructura con forma de árbol de forma dinámica y ejecutar el resultado desde un navegador.

Para realizar esta validación se implementa una versión deliberadamente reducida. El prototipo incluye un menú inicial y una [Escena](#) en la que el usuario puede crear y eliminar [nodos](#) organizados en una jerarquía. Al pulsar sobre un [nodo](#) aparece la opción de añadir un [nodo](#) hijo, que se coloca bajo el anterior y se conecta con el resto de la estructura. Esta prueba no incorpora todavía la lógica de un árbol de decisión. Su función es comprobar que la interacción principal, basada en modificar una estructura visual durante la ejecución, se puede implementar de forma estable.

6.1.1 Primeras [Escenas](#)

La primera [Escena](#) creada es el menú principal. En esta versión contiene un único botón, utilizado como punto de entrada hacia la prueba del árbol. El menú se incorpora desde el inicio

para establecer una navegación separada por módulos y dejar preparada la incorporación posterior de nuevas vistas. En la Figura 6.1 se muestra esta primera versión.

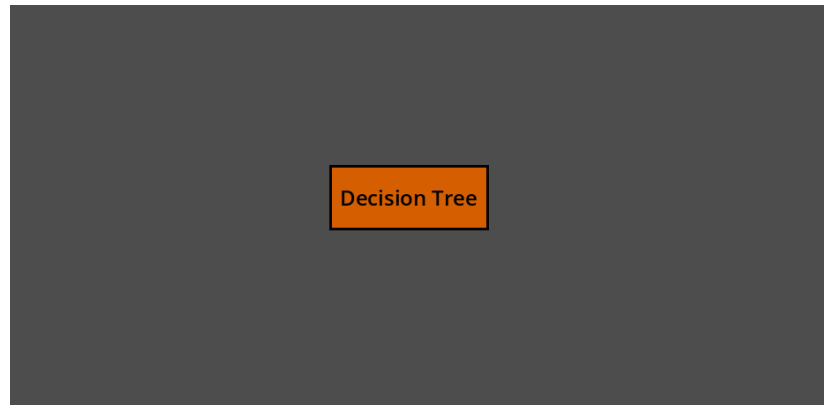


Figura 6.1: Versión preliminar del menú principal.

Dentro de la vista del árbol aparece un único **nodo** inicial. A partir de él pueden crearse nuevos **nodos** hijos, conectados por líneas y colocados bajo su respectivo padre. También se añade la posibilidad de borrar partes del árbol para comprobar que la estructura podía modificarse durante la ejecución sin perder legibilidad ni romperse.

La prueba se centra en validar que el usuario puede construir una jerarquía visual, que las conexiones se actualizaban de acuerdo con los cambios y que la pantalla sigue siendo manejable al aumentar el número de elementos. En la Figura 6.2 se muestra esta **Escena** inicial. La interfaz incluye también dos controles reservados para operaciones de carga y guardado.

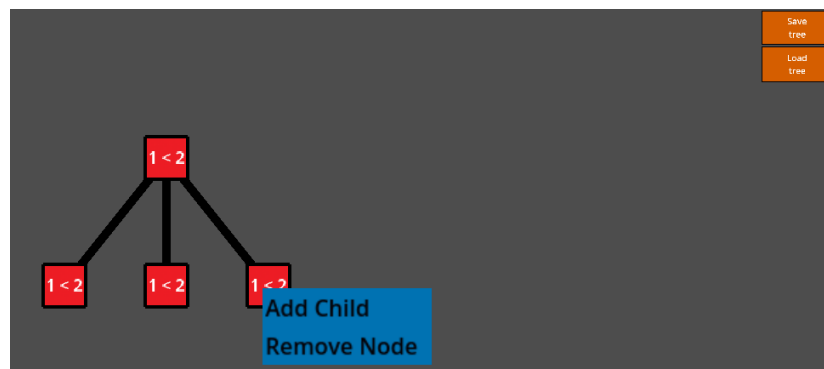


Figura 6.2: Primera **Escena** de prueba del árbol. Arriba a la derecha hay dos botones para guardar y cargar un árbol ya creado. Abajo a la izquierda aparece una estructura con forma de árbol, con **nodos** rojos y texto de ejemplo conectados por líneas negras. Sobre uno de los **nodos** se muestra el menú que permite añadirle un hijo o eliminarlo.

6.1.2 Exportación web con GitHub Pages

Una vez verificado el funcionamiento de la prueba interactiva en el entorno de desarrollo de Godot, se evalúa su despliegue en un navegador web. Esta comprobación es necesaria porque la aplicación está concebida para utilizarse a través de una interfaz web y debe ejecutarse correctamente fuera del editor.

La exportación del proyecto al formato web se realiza utilizando las herramientas proporcionadas por Godot. El motor genera una plantilla HTML y los archivos necesarios para ejecutar la aplicación dentro de un *canvas* en el navegador. Estos archivos forman una versión distinta del proyecto editable y contienen el resultado preparado para ser servido como página estática y cargado desde un entorno web real.

Tras generar la exportación, se publica el prototipo mediante GitHub Pages. Para ello se configura el repositorio de forma que los archivos exportados queden disponibles desde una dirección pública. El objetivo de realizar esta prueba tan pronto en el desarrollo es no retrasar la solución de problemas que puedan surgir a raíz de la transformación del proyecto a HTML5 o de su compatibilidad con GitHub Pages.

La prueba valida el flujo de trabajo previsto, basado en desarrollar en Godot, exportar la versión HTML5 y publicar el resultado como una aplicación accesible desde un enlace. Esta validación temprana tiene un peso importante en el desarrollo, porque confirma que la herramienta puede llegar al usuario sin instalación local ni infraestructura de servidor propia. También reduce el riesgo de descubrir problemas de despliegue cuando la implementación de los algoritmos ya esté avanzada.

Desde este primer prototipo, GitHub Pages queda integrado como parte del proceso de desarrollo. Cada versión exportada puede probarse en las mismas condiciones en las que se usará la aplicación final, lo que permite detectar problemas de carga, navegación o acceso a recursos antes de que afecten a módulos más complejos. Así, el despliegue web deja de ser una comprobación aislada y pasa a formar parte de la validación continua de la aplicación.

6.2 Árbol de decisión: versión 0.2

En esta iteración, la aplicación integra un árbol de decisión basado en ID3 como su primer algoritmo de aprendizaje. El trabajo de esta fase consiste en pasar de una estructura editable manualmente a una construcción generada por el algoritmo, con representación gráfica, avance paso a paso y explicación de las decisiones tomadas durante el entrenamiento. Esta incorporación requiere coordinar el cálculo de la siguiente división, la estructura lógica del árbol y la *escena* visible para el usuario. El funcionamiento formal de ID3 y del criterio de entropía se recoge en la Sección 5.1; en esta Sección se describe cómo se integra ese algoritmo en la aplicación.

6.2.1 Primera arquitectura del árbol

El primer cambio consiste en sustituir la *Escena* de prueba por otra preparada para representar árboles generados por el algoritmo. La prueba inicial valida la creación y eliminación manual de *nodos*, una interacción insuficiente para representar un entrenamiento real. El árbol de decisión puede crecer varios niveles y abrir varias ramas en poco espacio, de manera que una pantalla fija deja de ser adecuada al aumentar el número de *nodos*. Por esta razón se incorpora una zona desplazable dentro de la interfaz, que permite recorrer el árbol cuando su tamaño supera el área visible.

También se revisa la distribución de los *nodos* en el árbol. En la versión manual basta con colocar cada hijo bajo su padre, ya que el objetivo es comprobar la interacción básica. Durante el entrenamiento algorítmico se necesita una disposición estable, en la que los hijos aparezcan por debajo, el padre quede centrado respecto a ellos y el árbol se reorganice cada vez que se añade una rama. Para ello se introduce un cálculo de posiciones basado en la profundidad de cada *nodo* y en su orden dentro de la estructura. Este criterio evita que las ramas se acumulen en la misma zona y mantiene una lectura jerárquica clara durante las pruebas de entrenamiento.

En esta arquitectura se separa la información lógica del árbol de su representación visual. En cada *nodo* visible se conserva un identificador, sus relaciones dentro de la estructura, su profundidad y la información asociada al paso correspondiente, como el atributo de decisión o la etiqueta final. Al mismo tiempo, se mantiene un diccionario interno para localizar cada *nodo* por su identificador. Con esta organización se diferencian las tareas de la *Escena* de Godot, encargada de dibujar, animar y recibir eventos, de las operaciones sobre la estructura lógica, como recorrer el árbol, recalcular posiciones, eliminar subárboles y evaluar ejemplos en fases posteriores.

Las conexiones entre *nodos* se estructuran a nivel interno como elementos independientes. Un *Nodo* padre de todas se encarga de organizar el dónde dibujarlas. Cada conexión representa una rama del árbol y puede mostrar el valor del atributo que conduce desde el *nodo* padre hasta el hijo. Con esta separación, en los *nodos* se muestran decisiones o clasificaciones, y en las conexiones se indica el valor seguido en cada rama.

Con la base visual preparada, se añade el modo automático como estado interno de la *Escena* del árbol. En el modo manual se mantiene la interacción directa del usuario con los *nodos*, incluyendo su creación y eliminación. En el modo automático se delega la construcción del árbol en el algoritmo ID3, que genera la estructura a partir del conjunto de datos seleccionado. En este modo, el avance del entrenamiento se controla paso a paso, sin añadir ramas manualmente. En la primera versión de la interfaz se incluye un botón para iniciar el proceso y otro para avanzar al siguiente paso, como se muestra en la Figura 6.3.

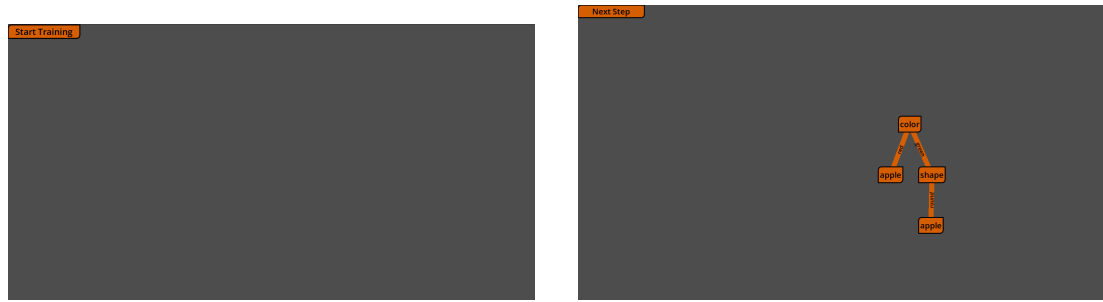


Figura 6.3: Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.

La construcción paso a paso se apoya en una separación de responsabilidades basada en el patrón Model-View-Controller [16]. El algoritmo calcula qué tipo de **nodo** debe crearse en cada momento y emite la información necesaria para hacerlo (Model). El controlador recibe esa información (Controller) y la traduce a cambios en la **Escena** (View), creando o actualizando los **nodos** visibles. Con esta división, la lógica de aprendizaje queda aislada de los elementos gráficos de Godot, y la vista representa el resultado de cada decisión. La Figura 6.4 resume esta relación entre el algoritmo, el controlador y las partes visibles de la **Escena**. Este esquema general es la base de la implementación de la red neuronal, detallada en la Sección 6.3.

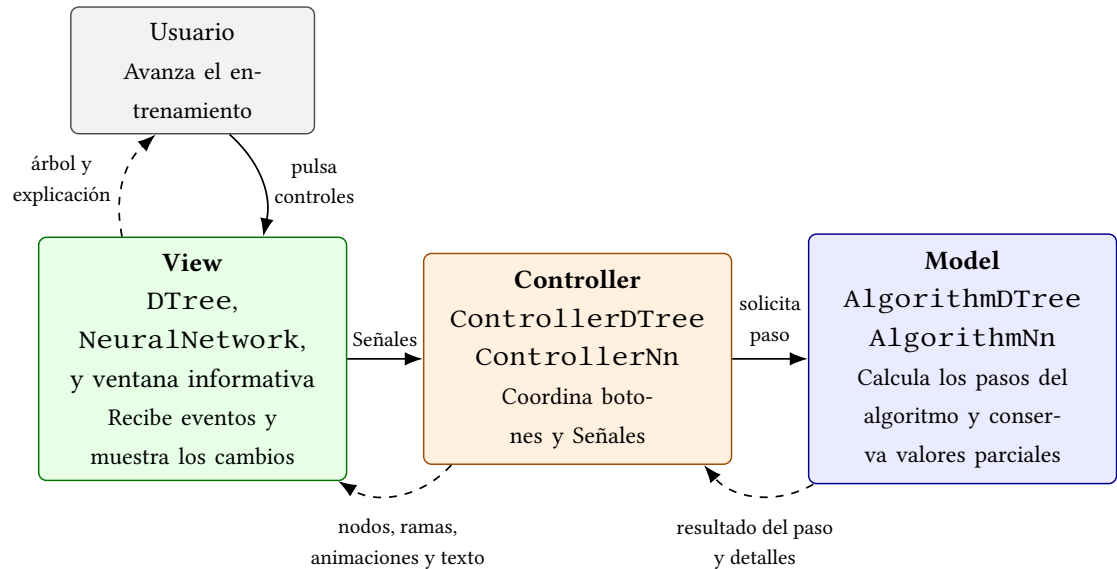


Figura 6.4: Separación de responsabilidades aplicada al árbol de decisión mediante el patrón Model-View-Controller.

El algoritmo no construye todo el árbol en una sola ejecución. El trabajo pendiente se almacena como una serie de contextos, cada uno con los datos necesarios para el cálculo de

los cambios en el árbol y la creación de los siguientes contextos. Con cada pulsación se extrae uno de esos contextos, se decide si debe convertirse en una hoja o en un **nodo** interno y se actualiza la **Escena**. Esta organización incremental permite explicar la construcción **nodo a nodo** sin ejecutar todo el entrenamiento de una vez.

Las primeras pruebas se realizan con un conjunto de datos pequeño y controlado. Su finalidad es comprobar que el algoritmo crea la raíz, abre ramas, coloca los **nodos** en posiciones coherentes y mantiene sincronizados el estado interno y la vista. En esta fase queda fijado el patrón principal del resto del desarrollo del árbol. El algoritmo decide, el controlador traduce esa decisión y el árbol visible se reorganiza tras cada paso.

6.2.2 Reorganización de la **Escena** y primeros elementos explicativos

Cuando el modo automático empieza a funcionar, la **Escena** se reorganiza para retirar elementos provisionales y adaptar la vista a una construcción guiada por el algoritmo. La **Escena** del árbol pasa a integrarse con los contenedores de interfaz de Godot, lo que permite combinar botones, paneles y zonas desplazables dentro de una misma estructura. La reorganización también afecta al centrado del árbol. En lugar de desplazar la **Escena** completa, se recalculan las posiciones internas de los **nodos** y el área mínima del contenedor desplazable. De esta forma, el árbol permanece centrado mientras cabe en pantalla y conserva la posibilidad de recorrerse cuando su tamaño supera el área visible.

La vista del árbol queda organizada como un flujo guiado. Al entrar en ella se muestra un panel inicial desde el que se escoge entre creación manual y creación algorítmica. Cada opción activa controles distintos, de manera que las acciones de edición directa y las acciones del entrenamiento automático quedan separadas. En la Figura 6.5 se muestra este panel inicial.



Figura 6.5: Panel inicial de la **escena** del árbol para escoger entre creación manual y creación algorítmica.

En esta etapa se revisa también la apariencia de los **nodos**. Los **nodos** internos, que representan preguntas sobre un atributo, y las hojas, que representan una clasificación final, reciben estilos visuales distintos, como se aprecia en la Figura 6.6. Esta diferencia separa visualmente las decisiones pendientes de las clasificaciones ya alcanzadas. Los **nodos** creados por el algoritmo dejan de ser editables de forma manual, ya que forman parte del estado del entrenamiento y deben modificarse únicamente al avanzar o retroceder por los pasos calculados. En el mismo tramo se traduce la interfaz al castellano. Los controles principales pasan a mostrar los textos “Empezar entrenamiento” y “Siguiente paso”, como se aprecia en la Figura 6.6. Con ello, la vista del árbol mantiene la misma lengua que el resto de la aplicación y se ajusta al tono explicativo del módulo.

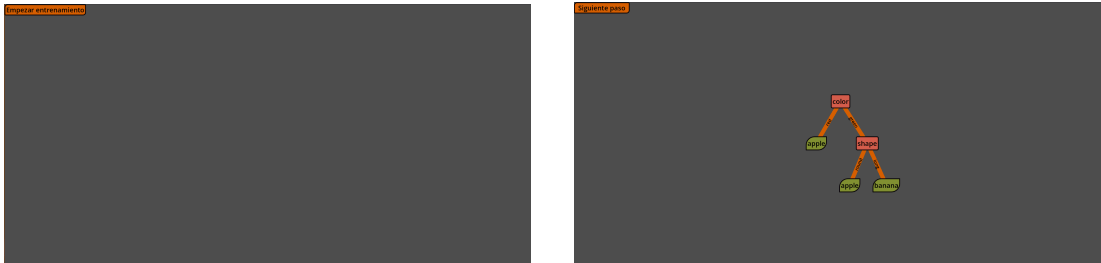


Figura 6.6: Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para **nodos** internos y hojas.

Una vez estabilizada la construcción visual, se incorpora la ventana informativa. Hasta este punto, la **escena** muestra qué elemento se añade al árbol en cada paso, con una explicación todavía limitada sobre el motivo de la decisión tomada. La ventana se introduce para acompañar cada avance con un texto sincronizado con el estado interno del algoritmo. Su primera versión contiene un mensaje inicial que indica el inicio del entrenamiento, como se observa en la Figura 6.7.

La ventana se implementa como un componente separado de la vista principal. Esta separación permite actualizar su contenido cada vez que el algoritmo procesa un contexto, sin mezclar la redacción de explicaciones con la creación de **nodos** en pantalla. El algoritmo prepara la información explicativa del paso actual y la ventana la recibe y transforma en texto integrado en la interfaz.

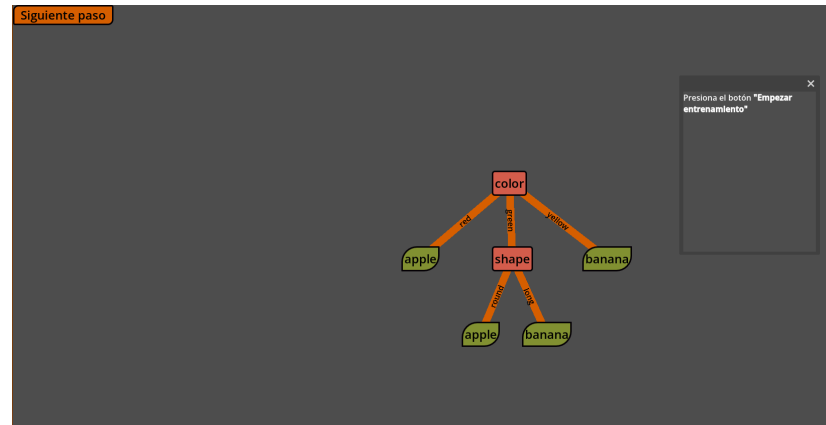


Figura 6.7: Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.

Los primeros textos explican la selección de un atributo en un **nodo** interno, la creación de hojas puras cuando todos los ejemplos de una rama comparten etiqueta y la creación de hojas por mayoría cuando ya no quedan atributos útiles para continuar dividiendo. Durante las pruebas se comprueba que la información interna del algoritmo necesita una preparación previa antes de mostrarse. Las listas de etiquetas o atributos conservan un formato propio de programación si se presentan tal como las maneja el código. Por ello, la ventana transforma esos valores en frases legibles, con enumeraciones y textos adaptados al caso concreto. Esta transformación mantiene la relación entre la estructura interna del programa y la explicación que se muestra en pantalla.

El criterio utilizado para elegir la siguiente división se incorpora inicialmente como una lista textual de valores calculados para cada atributo. Esta primera representación vincula el cálculo del algoritmo con una justificación visible dentro de la interfaz. A partir de este punto, la construcción del árbol queda acompañada por una explicación asociada a cada paso.

6.2.3 Animación de la construcción del árbol

Cuando la construcción paso a paso funciona de forma estable, aparece un problema de lectura visual. Una única pulsación puede crear un **nodo**, desplazar parte del árbol y dibujar varias ramas nuevas. La operación es correcta desde el punto de vista del algoritmo. El cambio en pantalla puede resultar brusco si todos los elementos se actualizan a la vez. Por esta razón se incorporan animaciones a la construcción.

Las primeras animaciones se aplican a las conexiones. Cada rama se dibuja progresivamente desde el **nodo** padre hasta el **nodo** hijo, de manera que el elemento incorporado en el paso actual queda identificado por la propia transición. Las etiquetas de las ramas también se integran en la conexión. La línea reserva un espacio para el texto y este se orienta siguien-

do la inclinación de la rama, con la corrección necesaria para que no quede invertido. Así se muestra el valor del atributo que conduce a cada hijo sin cargar los **nodos** con información secundaria.

Después se anima la aparición de los **nodos**. En vez de mostrarse de golpe, los **nodos** nuevos aparecen mediante una transición de opacidad. Este efecto cumple una función visual y funcional, ya que diferencia el elemento incorporado en el paso actual de los **nodos** que ya estaban presentes.

La introducción de animaciones requiere revisar la actualización de conexiones. En una versión temprana, al recalcular el árbol se redibujaban todas las ramas. Como consecuencia, una conexión antigua volvía a animarse cada vez que se añadía una nueva. La solución consiste en conservar las conexiones existentes, actualizar su posición cuando el árbol se re-coloca y animar únicamente las conexiones creadas en el paso actual. Con esta modificación, el movimiento de la **Escena** queda asociado al cambio producido por la última acción.

En esta misma etapa se añaden los **nodos** pendientes. Cuando el algoritmo crea un **nodo** interno, ya conoce los valores del atributo elegido y, por tanto, las ramas que saldrán de él. Todavía queda por resolver si cada rama terminará en una hoja o en otro **nodo** de decisión. Hasta este momento, esa información no se representaba en la vista, ya que las ramas se dibujaban únicamente cuando el siguiente **nodo** quedaba definido. Para representar el estado real del árbol en cada instante, se incorporan **nodos** temporales con texto provisional y un estilo propio que los diferencia. Así, las conexiones se dibujan cuando el algoritmo ya contempla una rama, antes de que se calcule su contenido definitivo. Los **nodos** pendientes muestran que una rama ya está prevista y que su contenido se resolverá en un paso posterior. Cuando el algoritmo procesa esa rama, el **nodo** temporal se transforma en el **nodo** definitivo, con su atributo o su etiqueta final. La transformación también se anima mediante una reducción de opacidad, un cambio de texto y tema visual, y una nueva aparición del **nodo**. De esta manera, la vista representa la sustitución de una posibilidad abierta por una decisión ya calculada. El aspecto de estos **nodos** pendientes se muestra en la Figura 6.8.

También se prueba una forma de consultar información asociada a **nodos** ya procesados. Al pasar por determinados **nodos**, el texto podía cambiar para mostrar un valor numérico vinculado a la decisión correspondiente. La solución permitía consultar información sin recargar visualmente todo el árbol.

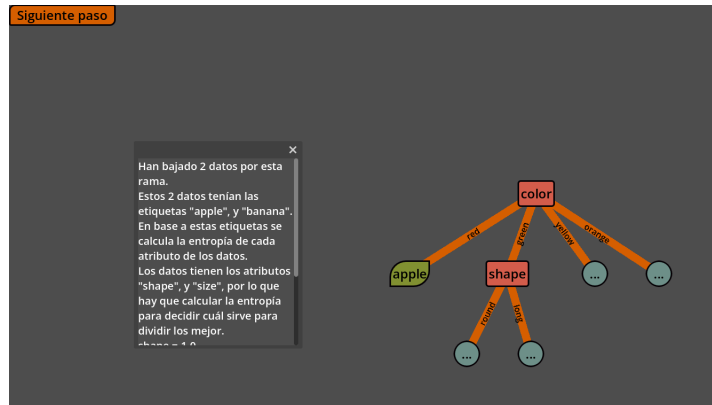


Figura 6.8: Entrenamiento del árbol en el que se aprecian los **nodos** pendientes.

6.2.4 Gráficas y casos explicativos en la ventana

La ventana informativa evoluciona a medida que los pasos del árbol incorporan más información. El texto plano permite explicar los primeros casos. Cuando el algoritmo compara varios atributos y selecciona uno de ellos, la explicación requiere una representación complementaria. En esta fase se combina la explicación escrita con un apoyo visual que representa la comparación de valores calculados.

El cambio principal es la incorporación de una gráfica de barras, mostrada en la Figura 6.9. La gráfica representa la métrica calculada para cada atributo, basada en la entropía y utilizada por el algoritmo como ganancia de información, tal como se define en la Sección 5.1. La representación visual permite comparar qué atributo reduce en mayor medida la incertidumbre de los datos y justifica su uso para abrir las ramas del **nodo** actual.

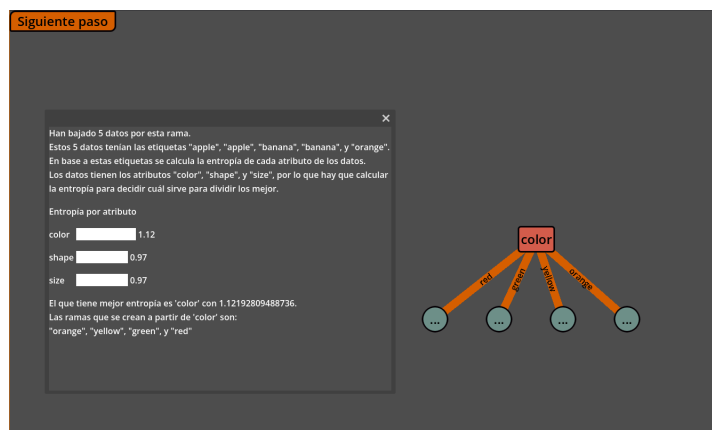


Figura 6.9: Ventana informativa en la que se dibuja la gráfica de la métrica basada en la entropía para cada atributo.

Para integrar la gráfica, la ventana se reorganiza como una zona desplazable con varios

elementos. El texto introduce los datos que llegan al **nodo**, las etiquetas presentes y los atributos disponibles; la gráfica ocupa la parte central de la explicación, y el cierre describe el atributo elegido junto con las ramas que se van a crear. Esta estructura sitúa la gráfica dentro de la explicación del paso actual y mantiene la continuidad entre los datos de entrada, el cálculo de la métrica y la decisión tomada.

El cambio también modifica la forma en la que se preparan los datos explicativos. Los valores de la métrica dejan de generarse únicamente como frases ya formateadas y se conservan como números. La misma información puede usarse entonces para dos fines: construir el texto de la explicación y alimentar la gráfica de barras. Esta separación mantiene la ventana sincronizada con el cálculo real del algoritmo y evita duplicar la lógica de la métrica en la interfaz.

Durante las pruebas aparece un caso relevante, en el que varios atributos pueden obtener el mismo valor máximo de la métrica. Si la aplicación escoge uno de ellos sin explicar la situación, la decisión puede interpretarse como arbitraria. Para cubrir este caso se añade una explicación específica de empate. La ventana indica que hay varias alternativas equivalentes y que el algoritmo continúa con una de ellas para poder seguir construyendo el árbol. También se revisan los textos asociados a las hojas. Las primeras redacciones estaban orientadas a conjuntos pequeños, donde en algunos casos solo llegaba un ejemplo a cada resultado final. Al probar datos más variados, se generaliza la explicación para hojas creadas a partir de varios ejemplos con la misma etiqueta y para hojas creadas por mayoría. La ventana describe así el motivo de una clasificación final sin depender del tamaño exacto del subconjunto que llega a la rama.

Este proceso de ajuste se repite durante toda la fase del árbol. Cada nuevo conjunto de datos revela situaciones que la explicación inicial no cubría de forma suficiente, como ramas puras, atributos agotados, empates, varios ejemplos con la misma clasificación o decisiones que crean **nodos** pendientes. La ventana incorpora esos casos para evitar que una decisión del algoritmo quede sin justificación visible dentro de la interfaz.

6.2.5 Primer panel de selección de datos

Hasta este momento, los datos de entrenamiento están predefinidos dentro del propio proyecto. Esta decisión facilita las primeras pruebas, porque permite repetir siempre el mismo ejemplo y centrar el esfuerzo en el algoritmo y la visualización. Una vez la construcción del árbol está asentada, esa limitación reduce la capacidad de observar cómo cambian las decisiones al modificar los datos de entrada.

El primer paso para resolverlo consiste en probar un segundo conjunto preparado manualmente. Este nuevo ejemplo incluye atributos y etiquetas distintos, y produce ramas más profundas, hojas por mayoría y casos con menos atributos disponibles. La prueba permite

comprobar que el algoritmo no depende de nombres concretos de columnas y que los textos de la ventana siguen siendo válidos con datos diferentes.

A partir de ahí se crea el panel de selección que se aprecia en la Figura 6.10. Su función es guiar la preparación del entrenamiento mediante la elección de una fuente de datos, la validación de la selección y la activación posterior del proceso. La primera opción permite usar un conjunto incluido en la aplicación, mientras que la segunda prepara la carga de archivos CSV.

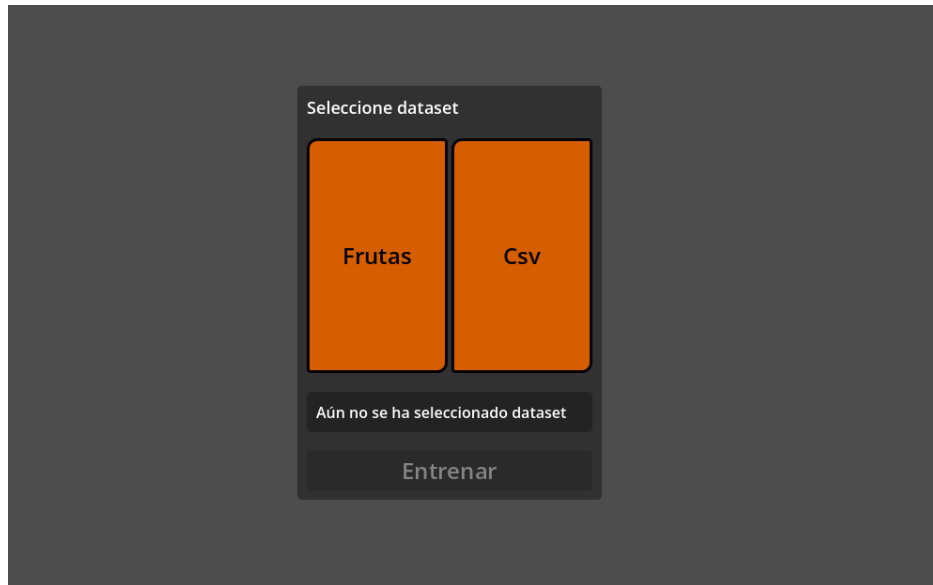


Figura 6.10: Menú que aparece tras seleccionar el modo algorítmico para escoger el conjunto de datos de entrenamiento.

La aparición de este panel cambia el flujo del modo algorítmico. Antes de esta incorporación, el entrenamiento comenzaba directamente al activar el modo automático. A partir de este cambio, la selección de datos genera un mensaje de confirmación o error y solo una selección válida permite avanzar por la construcción del árbol. La vista del árbol y la ventana informativa permanecen ocultas hasta que existe una selección válida, lo que evita mostrar contenido sin contexto.

La comunicación entre el panel de selección, el controlador y la vista se resuelve mediante Señales. Cuando el usuario confirma un conjunto de datos, el panel emite la información seleccionada y el controlador inicia el algoritmo con esos datos y atributos. Esta solución encaja con la organización de Godot, ya que permite que componentes situados en partes distintas de la *Escena* colaboren sin depender directamente unos de otros.

En esta etapa también se flexibiliza la carga de CSV para que la columna objetivo pueda tener un nombre distinto al prefijado desde el código, *clase*, al ampliar la lista de nombres re-

conocidos automáticamente como columna objetivo. De esta forma, el árbol puede entrenarse con conjuntos que usen nombres como *class* o *label*, siempre que el conjunto tenga una única columna objetivo.

6.2.6 Carga de archivos y navegación por pasos

Tras introducir el panel, se completa la carga de CSV para que los archivos seleccionados puedan alimentar el entrenamiento. La lectura se plantea de forma delimitada al tipo de datos que maneja esta versión del árbol: tablas simples, con una primera fila de cabeceras, varias filas de ejemplos y valores categóricos. La aplicación ignora filas vacías, transforma cada fila en un diccionario de valores y separa la columna objetivo del resto de atributos mediante nombres habituales o mediante la diferencia entre columnas de entrada y salida.

El panel informa al usuario del resultado de la carga. Si el archivo puede leerse y la estructura es válida, se activa el botón de entrenamiento. Si aparece un problema, se muestra un mensaje de error y se impide continuar. Esta comprobación evita que el algoritmo empiece con datos incompletos o sin una etiqueta identificable.

Con la selección de datos integrada, el entrenamiento empieza a organizarse en fases. La selección y confirmación del conjunto de datos preceden a la aparición del menú del árbol, desde el que se avanza por la construcción. Esta separación hace explícita la dependencia entre el conjunto de entrada y la forma final del árbol.

El siguiente cambio relevante es la navegación hacia atrás. Hasta entonces, el entrenamiento solo avanzaba, lo que obligaba a reiniciar el proceso si el usuario quería revisar una decisión anterior. En una herramienta orientada a la visualización del aprendizaje, el retroceso permite revisar por qué se abrió una rama, por qué apareció una hoja o qué atributo se escogió en un **nodo** interno. Por ello se añade el botón “Paso previo”, visible en la Figura 6.11.

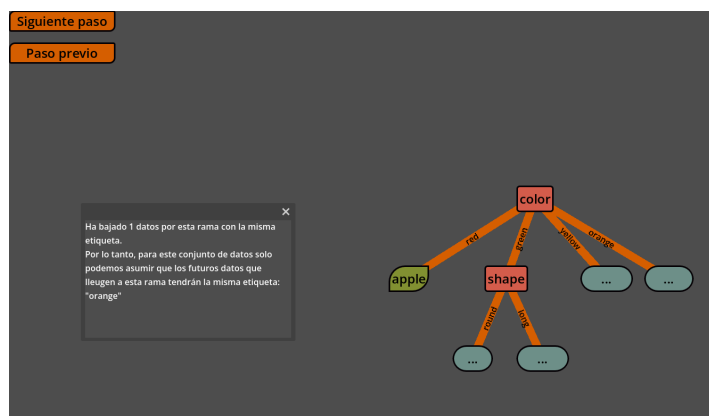


Figura 6.11: Vista del árbol con el botón para retroceder al paso anterior.

El retroceso se apoya en la misma idea que permite avanzar por pasos. El algoritmo man-

tiene una colección de contextos pendientes y otra de contextos ya ejecutados. Al avanzar, un contexto pasa de pendiente a ejecutado. Al retroceder, el último contexto ejecutado vuelve a quedar pendiente y la *Escena* debe deshacer su efecto visual. Esta estructura evita reconstruir todo el árbol desde cero cada vez que se pulsa “Paso previo”.

El retroceso requiere tratar varios casos para mantener la coherencia entre algoritmo y vista. Si se deshace un *nodo* interno, también deben retirarse los contextos hijos y los *nodos* visuales que dependen de él. Si se deshace una rama que estaba representada por un *nodo* pendiente, se restaura ese estado provisional en lugar de eliminar toda la indicación visual. Si el usuario retrocede hasta el inicio, la raíz desaparece y el árbol debe quedar vacío. Durante las pruebas se detecta un fallo en ese último caso. Al volver al paso cero, podían quedar conexiones visibles después de eliminarse los *nodos*. La corrección consiste en recalcular también las conexiones cuando el árbol queda sin raíz o sin *nodos*. Con ello, retroceder hasta el inicio deja la vista en un estado consistente y permite avanzar de nuevo sin restos visuales de la ejecución anterior.

6.2.7 Primera evaluación visual del árbol

El último avance de esta fase es la primera versión del modo de evaluación. Hasta este punto, la aplicación representa cómo se construye el árbol. El siguiente objetivo consiste en mostrar cómo se utiliza ese árbol ya entrenado para clasificar nuevos ejemplos. La evaluación se plantea de forma visual: cada dato de prueba se representa como un marcador que se desplaza por la estructura siguiendo las ramas aprendidas.

Para introducir este modo se añaden nuevos controles al panel del algoritmo. Durante el entrenamiento, la evaluación permanece desactivada. Cuando el árbol termina de construirse, el botón de avanzar se bloquea y se habilita la opción de evaluar. Al activarla, se reutiliza el panel de selección de datos para escoger un conjunto de prueba. Así se separan la selección de datos de entrenamiento y la selección de datos destinados a comprobar el árbol resultante.

La evaluación utiliza una representación sencilla de cada ejemplo, como se aprecia en la Figura 6.12. Cada dato se dibuja como un cuadrado pequeño que entra en el árbol desde la raíz al pulsar el botón correspondiente. El recorrido se calcula leyendo el atributo de cada *nodo* interno y buscando la rama cuyo valor coincide con el dato evaluado. Cuando la pieza llega a una hoja, el recorrido termina y queda representada la clasificación obtenida.

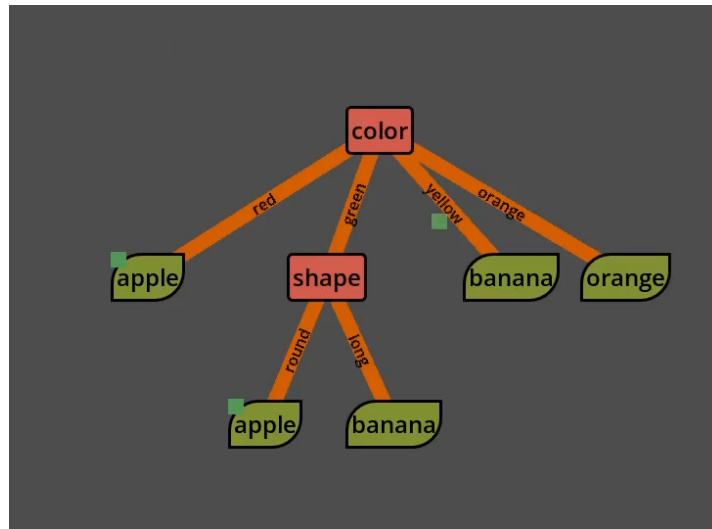


Figura 6.12: Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.

Para evitar dependencias innecesarias con el árbol en construcción, la evaluación toma una copia de la información relevante de los **nodos** visibles, formada por la posición, la profundidad, el atributo, la etiqueta, el valor de rama y el padre. Con esa información puede mover los datos por la **Escena** y decidir el siguiente **nodo** sin alterar la estructura entrenada. El movimiento entre **nodos** se anima para representar el recorrido como una secuencia de decisiones, en lugar de mostrar únicamente el resultado final.

Esta primera versión del modo de evaluación conserva una representación deliberadamente simple. Los datos se muestran con formas básicas y cada pulsación introduce un ejemplo que recorre el árbol hasta una hoja. Las mejoras posteriores se orientan a agrupar visualmente los ejemplos bajo las hojas, enriquecer la explicación de cada decisión e introducir una navegación más detallada del recorrido. Esta prueba establece la base del modo de evaluación: representar visualmente cómo un ejemplo atraviesa las decisiones aprendidas hasta llegar a una clasificación.

Con este conjunto de cambios queda completada la primera versión significativa del árbol de decisión. El módulo permite entrenar un árbol categórico paso a paso, mantener una estructura lógica sincronizada con la **Escena**, mostrar ramas con valores, distinguir **nodos** internos, hojas y **nodos** pendientes, animar los cambios, explicar las decisiones con texto y gráficas, cargar datos sencillos, retroceder por la construcción y evaluar ejemplos nuevos de forma visual.

La arquitectura concreta que queda al final de esta fase se resume en la Figura ?? . El módulo se reparte entre los paneles de entrada, el controlador, el algoritmo, la estructura lógica y los elementos de explicación. El controlador coordina las acciones del usuario, solicita pasos a

`AlgorithmDTree` y aplica cada resultado sobre `DTree`. El algoritmo mantiene los contextos pendientes y ejecutados mediante `call_stack` y `called_stack`, y devuelve el diccionario `details` junto con el resultado del paso. La estructura lógica conserva identificadores, relaciones y posiciones, a partir de los cuales se actualizan los **nodos** visibles, las conexiones y la copia usada durante la evaluación.

A partir de aquí, el desarrollo continúa con el segundo algoritmo de la aplicación: la red neuronal.

6.3 Red neuronal: versión 0.3

Una vez completada la primera versión funcional del árbol de decisión, el proyecto incorpora la red neuronal multicapa, el segundo modelo previsto. El cambio de algoritmo implica también un cambio en el tipo de visualización necesaria. En el árbol, el entrenamiento construye una estructura que crece con nuevas ramas. En la red neuronal, la estructura se define antes de entrenar y el aprendizaje modifica valores numéricos asociados a esa estructura, principalmente pesos y sesgos. La formulación matemática del modelo, del forward pass y de la retropropagación se presenta en la Sección 5.2; aquí se describe la implementación interactiva desarrollada en Godot.

Por esta razón, el desarrollo de la red empieza por resolver un problema distinto al del árbol. Antes de mostrar el forward pass o el backward pass, la aplicación debe representar una arquitectura formada por capas, neuronas y conexiones densas, con una correspondencia clara con el modelo lógico que se va a entrenar. La red visible funciona como soporte para los cálculos posteriores, en los que cada neurona debe poder resaltarse, y cada conexión debe poder actualizarse.

La **Escena** de red neuronal sigue el mismo patrón de diseño Model-View-Controller utilizado en el árbol de decisión, mostrado en la Figura 6.4. La lógica matemática, la representación visual y los elementos informativos se separan en componentes distintos, comunicados mediante Señales. Esta separación permite que el algoritmo avance paso a paso y que la **Escena** represente el estado correspondiente en cada momento. El resultado de esta versión es el núcleo funcional de la red, que cuenta con una arquitectura configurable, adaptada al conjunto de datos, entrenable de forma incremental y capaz de mostrar tanto el flujo de datos hacia delante como el retorno del error.

6.3.1 Primera arquitectura visual de la red

El primer paso consiste en añadir una **Escena** propia para redes neuronales y conectarla con el menú principal. Esta cuenta con sus propios **Nodos** de interfaz y sus propios objetos visuales, de manera que el desarrollo del segundo algoritmo no quede mezclado con la **Escena**

del árbol.

La representación inicial de la red se construye a partir de capas, neuronas y conexiones. Las capas se colocan en horizontal para mostrar el recorrido natural de los datos, desde la entrada situada a la izquierda hasta la salida situada a la derecha. Dentro de cada capa aparecen las neuronas, representadas como botones interactivos. Esta elección permite resaltarlas durante el entrenamiento y reutilizarlas más adelante como puntos de consulta de información. Y al igual que en el árbol, las conexiones se crean como hijas de un **Nodo** padre que gestiona cómo se dibujan.

La red parte de una estructura mínima formada por una capa de entrada y una capa de salida. Esta decisión proporciona una base visual sobre la que se incorporan después capas ocultas y controles de configuración. Las capas ocultas se incorporan después, cuando el menú de creación permite configurar la arquitectura. Desde esta primera fase, la vista se concibe como una estructura dinámica en la que el número de capas y neuronas puede cambiar, y la **Escena** debe reconstruir las conexiones para seguir representando una red coherente.

Las conexiones se dibujan entre capas consecutivas y siguen una estructura densa. Esto significa que cada neurona de una capa queda conectada con todas las neuronas de la capa siguiente. En una primera lectura, estas líneas sirven para mostrar la topología de la red. Más adelante pasan a representar también información del modelo, ya que su color y grosor dependen del peso asociado. Con ello, la **Escena** deja preparado el vínculo entre la arquitectura visible y los valores numéricos que el entrenamiento modificará. En la Figura 6.13 se muestra esta primera representación visual.

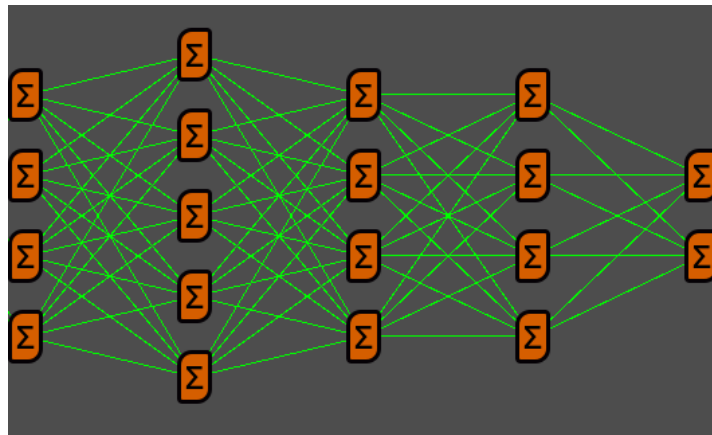


Figura 6.13: Primera arquitectura visual de la red neuronal, formada por capas, neuronas y conexiones densas entre capas consecutivas.

Esta primera arquitectura también fija una separación importante entre la red que se ve y la red que se calcula. La **Escena** se encarga de colocar capas, neuronas y conexiones, mientras que el estado lógico conserva la estructura y los valores que definen el modelo. Mantener

ambas partes separadas evita que la visualización sea la única fuente de información de la red. La vista puede reconstruirse, animarse o actualizar sus conexiones sin perder la referencia al modelo lógico que después usará el algoritmo de entrenamiento.

6.3.2 Configuración de la arquitectura y representación lógica

Una vez definida la [Escena](#) básica de la red, el siguiente paso consiste en permitir que el usuario configure su arquitectura desde la propia aplicación. Para ello se crea un menú de construcción que controla el número de neuronas de entrada, el número de neuronas de salida, la cantidad de capas ocultas, las neuronas de cada capa oculta y las funciones de activación asociadas, como se aprecia en la Figura 6.14. Con este panel, la arquitectura puede editarse dentro de unos límites conocidos.

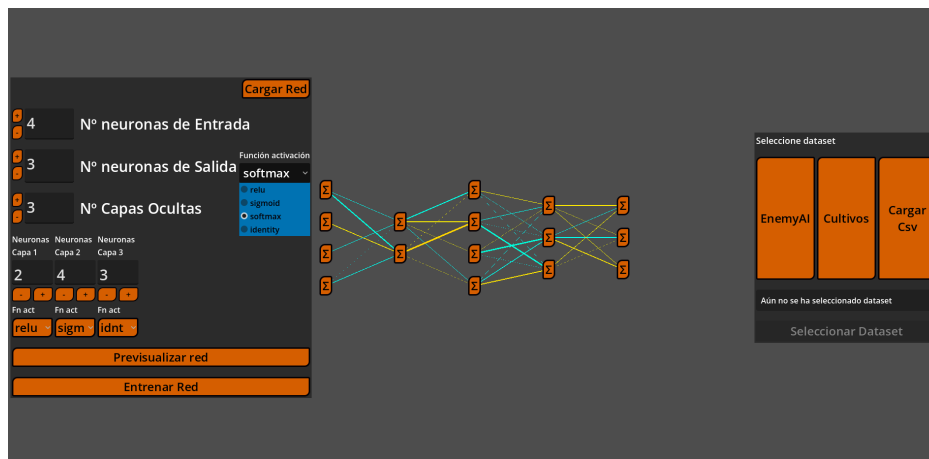


Figura 6.14: A la izquierda el menú de creación de la red neuronal.

Estos límites son necesarios para mantener una visualización manejable durante la interacción. Por ello se establecen máximos para el número de capas y para el número de neuronas por capa, de forma que la red no crezca hasta ocupar un espacio que dificulte su lectura. También se validan los valores introducidos en el menú antes de aplicarlos, con el fin de evitar cantidades negativas de neuronas o capas vacías.

La representación lógica de la red se organiza mediante una convención de identificadores. La capa de entrada usa el identificador 0, las capas ocultas usan enteros positivos y la capa de salida usa el identificador -1. Esta decisión permite tratar la salida como un elemento especial y, al mismo tiempo, recorrer las capas ocultas en orden natural. También facilita que las Señales de Godot indiquen con claridad qué capa debe añadir neuronas, eliminarlas, cambiar su activación o actualizar sus conexiones. El modelo lógico conserva, además del número de capas, las matrices de pesos que conectan cada capa con la anterior y la función de activación que debe aplicarse en cada una.

La capa de entrada recibe un tratamiento distinto en este contexto. Su función es mostrar los valores que entran en la red, por lo que no necesita una transformación aprendida como las capas posteriores. Esto se traduce en pesos de valor 1, sin actualización durante el entrenamiento, y en una función de activación lineal. En la estructura lógica se mantiene como parte de la arquitectura para que la vista pueda dibujar sus neuronas y conectarlas, mientras que la parte entrenable comienza en las capas que reciben esos valores.

La modificación de una capa obliga a actualizar otras partes de la red lógica. Si cambia el número de neuronas de una capa, también cambia el número de pesos que necesita la capa siguiente, ya que cada una de sus neuronas debe recibir una conexión desde todas las neuronas anteriores. Si se añade una capa oculta, se crean sus pesos iniciales y se reajusta la capa de salida para conectarse con la nueva capa previa. Si se elimina una capa, se borran sus datos y se vuelve a enlazar la salida con la última capa que permanece en la arquitectura. De esta forma, el modelo conserva siempre una topología densa y consistente.

La representación visual debe seguir esos mismos cambios. Al añadir o eliminar capas y neuronas, el contenedor de conexiones densas recalcula los enlaces entre capas consecutivas y borra los que ya no son válidos. Esta corrección evita que, al insertar capas ocultas, permanezcan conexiones antiguas entre entrada y salida ajenas a la red real. Las actualizaciones se realizan de forma diferida cuando es necesario, para que Godot termine de crear o eliminar **Nodos** antes de que las líneas intenten consultar sus posiciones. Así se conserva el orden visual de la red durante los cambios de arquitectura.

Con esta organización, la arquitectura editable queda conectada con una representación lógica estable. El menú actúa como interfaz de configuración, la **Escena** traduce esa configuración a capas, neuronas y conexiones, y el modelo interno conserva los pesos y activaciones que después usará el algoritmo. Este paso deja preparada la red para una condición que se resuelve a continuación: adaptar automáticamente la arquitectura al conjunto de datos elegido.

6.3.3 Selección de datos y restricciones de la red

La configuración manual de la arquitectura queda condicionada por el conjunto de datos utilizado. Cada atributo usado como entrada necesita una neurona en la primera capa, y cada clase o variable objetivo necesita una representación en la salida. Por este motivo se incorpora un panel de selección de datos específico para la red neuronal. El panel ofrece conjuntos internos y carga de archivos CSV. Los conjuntos internos cubren los dos tipos de problema principales que soporta la aplicación: clasificación y regresión. La distinción entre ambos casos determina cómo se fija el número de neuronas de la capa de salida.

Cuando se carga un CSV, la aplicación muestra las columnas disponibles y permite seleccionar explícitamente una o varias como objetivo. Esta decisión flexibiliza la carga de datos respecto a las primeras pruebas, donde se dependía de nombres habituales como *class*, *label* o

target. El resto de columnas se tratan como atributos de entrada. En la Figura 6.15 se muestra este selector de columnas objetivo.

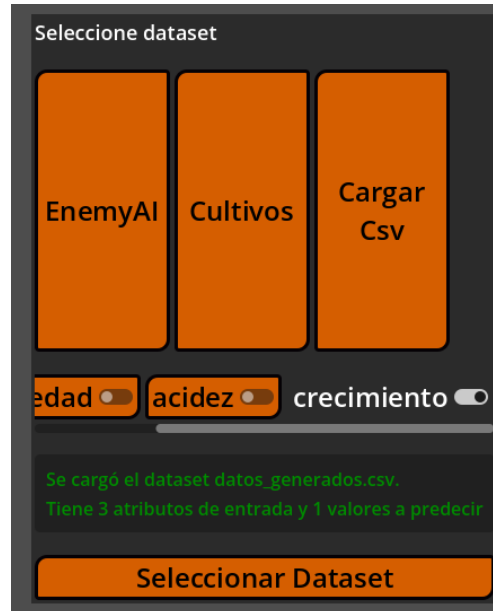


Figura 6.15: Selección de columnas objetivo para la red neuronal a partir de un conjunto de datos cargado desde un archivo CSV.

El panel analiza los tipos de las columnas seleccionadas para decidir cómo preparar el problema. Si hay una única variable objetivo categórica, o una única variable entera tratada como clase, el problema se considera de clasificación. Si las variables objetivo son numéricas continuas, el problema se considera de regresión. Los valores numéricos se normalizan cuando es necesario y las entradas categóricas se convierten a valores enteros para que puedan circular por la red. En clasificación las etiquetas textuales, y también las clases enteras que no forman una secuencia compacta, se codifican en identificadores contiguos. Así, una clase con valor original 10 queda asignada a un identificador interno compacto. La red solo necesita una neurona por clase real, y la aplicación conserva un decodificador para volver a mostrar los nombres originales en la interfaz. Con estas transformaciones, el conjunto de datos se adapta a las necesidades de entrada y salida de una red neuronal.

Con la información del conjunto de datos se construye un conjunto de restricciones para el menú de creación. La capa de entrada queda fijada al número de atributos seleccionados. La capa de salida queda fijada al número de clases o variables objetivo. La función de activación de salida también queda determinada por el tipo de problema: softmax para clasificación e identidad para regresión. Cuando esta restricción llega al menú, el selector de activación final se ajusta y queda bloqueado para evitar una configuración incoherente.

Mientras, el resto de la arquitectura sigue siendo configurable. El usuario puede modificar

capas ocultas, número de neuronas internas y funciones de activación intermedias, siempre dentro de los límites definidos en el menú. De esta manera, la aplicación combina libertad de experimentación con las restricciones mínimas que impone el conjunto de datos. La red creada siempre recibe vectores del tamaño correcto y produce salidas que pueden compararse con los valores esperados.

La selección de datos también mejora la lectura de la red visible. Las neuronas de entrada pasan a mostrar los nombres de los atributos, y las neuronas de salida muestran las clases decodificadas o las variables objetivo de regresión. Con estos nombres, cada extremo del modelo representa información reconocible del conjunto de datos y la lectura de la arquitectura es más explícita. Esta relación entre conjunto de datos y red visible se muestra en la Figura 6.16, donde las neuronas aparecen etiquetadas con información del conjunto seleccionado.

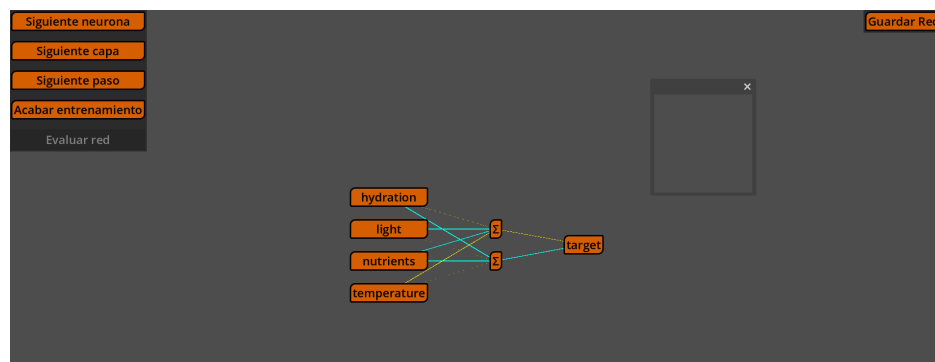


Figura 6.16: Red neuronal durante el entrenamiento, con nombres procedentes del conjunto de datos en las neuronas de entrada y salida.

Con esta dependencia entre datos y arquitectura, el flujo de la vista se reorganiza. Primero se selecciona el conjunto de datos y después se crea la red. Este orden evita que el usuario configure una arquitectura incompatible y, al mismo tiempo, permite que el menú aparezca ya inicializado con las entradas, salidas y activación final adecuadas. Una vez fijadas estas condiciones, la red queda preparada para conectarse con el algoritmo de entrenamiento incremental.

6.3.4 Algoritmo de entrenamiento incremental

Una vez que la arquitectura y los datos están definidos, hay que entrenar la red. Esta responsabilidad se concentra en un **Nodo** dedicado a procesar el algoritmo, separado de la vista. Su papel consiste en calcular el siguiente avance, actualizar el estado interno y emitir Señales para que la **Escena** represente el cambio correspondiente. La misma estructura que el algoritmo del árbol.

Al comenzar el entrenamiento, la **Escena** configura el algoritmo con toda la información

necesaria. Le entrega las filas del conjunto de datos, los atributos de entrada, las columnas objetivo y toda la información de la arquitectura de la red. Con esa información, el algoritmo prepara el orden de capas, reinicia sus cursores internos y deja lista la primera muestra. El mismo mecanismo sirve más adelante para inferencia, con una configuración que recorre la red sin actualizar pesos.

Una vez iniciado, el entrenamiento se organiza como una máquina de estados. Primero se carga una muestra del conjunto de datos y se colocan sus valores de entrada en la capa inicial. Después se ejecuta el forward pass, se compara la salida calculada con el objetivo esperado, se prepara el retorno visual del error y se ejecuta el backward pass. Al terminar una muestra, el algoritmo avanza a la siguiente usando los pesos que acaban de actualizarse. Este recorrido se repite hasta completar los datos previstos o hasta que se alcanza el estado final. Esta secuencia sigue el proceso formal descrito en la Sección 5.2, adaptado aquí a una ejecución observable paso a paso.

Para sostener este proceso, el algoritmo conserva varios diccionarios internos. Uno almacena las salidas activadas de cada capa, otro conserva las sumas ponderadas previas a la activación y otro guarda los deltas calculados durante la retropropagación. Estos valores tienen una doble utilidad: permiten continuar el cálculo sin repetir operaciones ya realizadas y sirven como fuente para las explicaciones mostradas en la ventana informativa.

El proceso de aprendizaje se controla desde un panel específico. El usuario puede avanzar una sola neurona, una capa completa, una muestra completa o terminar todo el entrenamiento pulsando diferentes botones. Cada opción llama al mismo proceso interno, cambiando únicamente hasta dónde debe avanzar antes de devolver el control a la interfaz. Así se mantiene una única lógica de entrenamiento y se ofrecen varios niveles de detalle para observar el proceso. La Figura 6.17 muestra este panel de control.

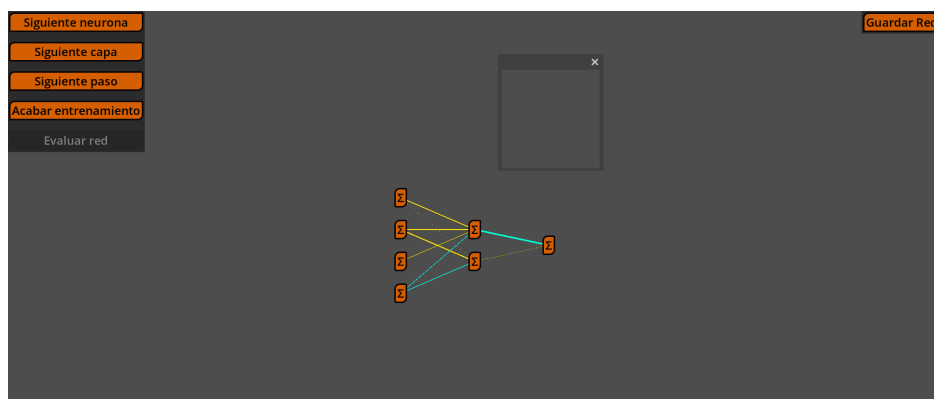


Figura 6.17: Panel de entrenamiento de la red neuronal con controles para avanzar con distintos niveles de granularidad.

La ejecución por muestras permite representar el aprendizaje como una secuencia de cam-

bios localizados. Cada fila del conjunto de datos produce una entrada concreta, una salida concreta, un error concreto y una actualización de pesos concreta. La siguiente fila utiliza la red modificada por la anterior, de modo que el entrenamiento queda representado como una sucesión de correcciones sobre el mismo modelo. En esta versión se realiza la actualización directamente después de cada muestra. Esta decisión reduce la complejidad visual del procedimiento y concentra la explicación en la relación entre predicción, error y ajuste de parámetros.

El algoritmo también coordina los elementos visuales que acompañan al entrenamiento. Al cargar una muestra, emite Señales para preparar los valores de entrada y los valores esperados. Durante el *forward pass* y el *backward pass*, emiten Señales para resaltar la parte de la red que se está procesando. Cuando un peso cambia, emite la actualización correspondiente para que la conexión visual modifique su grosor o color según el nuevo valor.

Al finalizar el entrenamiento, la interfaz cambia de estado. Los botones de avance quedan desactivados y se habilita la evaluación de la red. Este cierre evita que el usuario continúe modificando una ejecución ya terminada y marca el paso desde el aprendizaje hacia el uso del modelo entrenado.

6.3.5 Forward pass y visualización de datos

El *forward pass* es la fase en la que una muestra atraviesa la red desde la capa de entrada hasta la capa de salida. En la aplicación se ejecuta capa a capa y, dentro de cada capa, neurona a neurona. Esta granularidad permite observar qué parte de la red participa en cada instante del cálculo.

Al cargar una muestra, el algoritmo extrae sus valores de entrada siguiendo el orden de los atributos seleccionados. Ese vector se guarda como salida de la capa de entrada, ya que la primera capa entrenable usa esos valores como datos de partida. En paralelo, la [Escena](#) muestra esos valores junto a las neuronas de entrada mediante pequeños elementos visuales asociados a cada atributo. Al comenzar el cálculo, los valores se animan hacia la red y desaparecen, lo que representa la entrada de la muestra en el modelo.

Para cada neurona, el cálculo sigue la operación de una capa densa explicada en la [Sección 5.2](#). El algoritmo combina los valores de la capa anterior con los pesos de entrada de la neurona, incorpora su sesgo y aplica la función de activación configurada para la capa. Así, cada avance produce un valor intermedio que puede mostrarse y usarse como entrada de la capa siguiente.

La activación softmax requiere un tratamiento específico dentro de la implementación. Como sus salidas dependen del vector completo de la capa, primero se calculan todos los valores previos a la activación y después se aplica la función al conjunto completo. La explicación de la ventana informativa refleja este comportamiento para presentar softmax como

una operación de capa.

Durante el *forward pass* se guardan dos tipos de valores por capa: las sumas ponderadas previas a la activación y las salidas activadas. Esta caché permite que la explicación, el cálculo del error y el *backward pass* usen exactamente los mismos resultados que se obtuvieron al propagar la muestra hacia delante.

La salida de la red también se representa visualmente. Junto a la capa final aparece una columna de valores producidos por la red, y junto a ella aparece otra columna con el valor esperado. Cuando se calcula una neurona de salida, su valor se añade a la columna de salida mientras los valores anteriores de la misma muestra se mantienen visibles. De esta manera, la predicción se completa gradualmente y queda alineada con las clases o variables objetivo que se habían mostrado en la arquitectura, como se ve en la Figura 6.18.

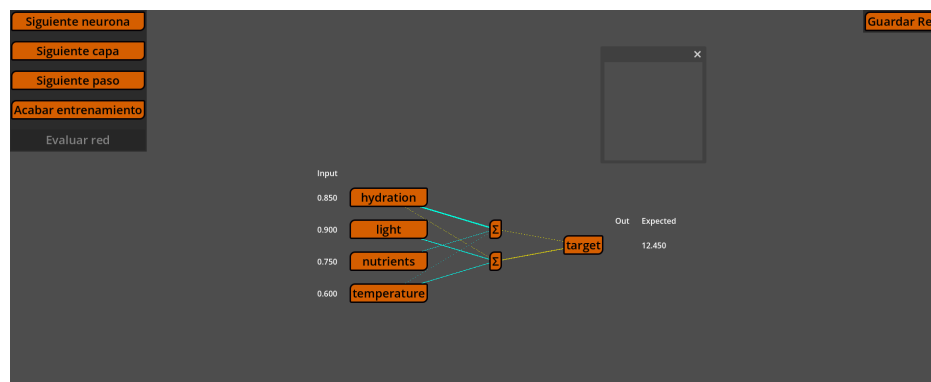


Figura 6.18: Red neuronal durante el entrenamiento, con la información del dato de entrada, su salida y su valor esperado.

El resaltado de neuronas y capas acompaña todo el proceso. Cada vez que se evalúa una neurona, esta cambia de estado visual y la ventana informativa recibe los datos necesarios para explicar el cálculo. Al terminar una capa, el vector de salidas activadas pasa a ser la entrada de la capa siguiente. Así se representa el recorrido completo de la muestra, desde los atributos iniciales hasta la predicción final.

Cuando termina la capa de salida, el flujo se bifurca según el modo de ejecución. En entrenamiento, el algoritmo pasa a comparar la salida con el objetivo esperado. En inferencia, la muestra termina en ese punto, porque evaluar una red entrenada consiste únicamente en propagar los datos hacia delante.

6.3.6 Cálculo del error y backward pass

Tras completar el forward pass, la red ya dispone de una salida para la muestra actual. El siguiente paso consiste en compararla con el objetivo esperado y convertir esa diferencia en

una señal de error que pueda volver por la red. La interfaz representa esta transición desplazando la columna de salida y transformando la columna de valores esperados en valores de error o delta. Así se marca visualmente el cambio entre producir una predicción y corregir el modelo a partir de ella.

El backward pass recorre las capas en orden inverso, empezando en la capa de salida, donde se conoce la diferencia entre lo calculado y lo esperado, y continuando hacia las capas ocultas. En cada neurona se calcula un delta, que resume cómo debe cambiar esa neurona para reducir el error de la muestra. Ese delta se usa después para actualizar los pesos que llegan a la neurona.

En la capa de salida hay varios casos, siguiendo las expresiones recogidas en la Sección 5.2. Cuando el problema es de clasificación y se usa softmax con entropía cruzada, el delta coincide con la diferencia vectorial entre la salida activada de la red y el vector esperado. Para otras combinaciones de pérdida y activación, el algoritmo calcula primero el gradiente de la pérdida respecto a la salida activada y lo combina con la derivada local de la función de activación. Con ello, la salida genera el primer conjunto de deltas que inicia la retropropagación.

Las capas ocultas reciben el error de la capa siguiente. Para cada neurona oculta se calcula cuánto ha contribuido a los deltas posteriores, ponderando esos deltas con los pesos de salida de la neurona. Después se tiene en cuenta la derivada de su propia activación. La explicación presenta la corrección de una neurona oculta a partir de su influencia en las neuronas de la capa posterior.

Una vez calculado el delta de una neurona, se actualizan sus pesos y su sesgo mediante descenso de gradiente. Cada peso se corrige en función del delta de la neurona, del valor que llega desde la capa anterior y de la tasa de aprendizaje configurada. El sesgo se actualiza con el propio delta, de forma coherente con la formulación de la Sección 5.2. La inclusión del sesgo permite que la red aprenda desplazamientos y relaciones que no pasan necesariamente por el origen.

Cada actualización se sincroniza con la representación lógica de la red. Los pesos y sesgos modificados se copian de vuelta al modelo compartido, y cada peso emite una Señal de actualización. El contenedor de conexiones recibe esa Señal y redibuja la línea correspondiente con el color y grosor derivados del nuevo valor. De esta manera, el backward pass se observa como un cambio visual del modelo mientras aprende.

Al completar la última capa oculta, la muestra queda terminada. El algoritmo guarda la información necesaria para explicar el paso, prepara la siguiente fila del conjunto de datos y repite el proceso con los pesos ya actualizados. El entrenamiento queda así estructurado como una secuencia de predicciones, errores y correcciones acumuladas sobre la misma red.

6.3.7 Ventana informativa de la red neuronal

La red neuronal necesita una ventana informativa más detallada que la del árbol, porque el aprendizaje depende de operaciones numéricas encadenadas. El resaltado de una neurona o el cambio de grosor de una conexión proporcionan una referencia visual del proceso. La ventana completa esa representación al explicar qué cálculo produce cada cambio. Para ello, recibe datos internos del algoritmo y los convierte en texto y fórmulas comprensibles.

La ventana se mantiene separada del algoritmo y de la [Escena](#). El algoritmo emite Señales con la información del paso actual, y la ventana decide cómo presentarla. Esta organización permite que el cálculo siga estando en la clase de entrenamiento, que la red visible se centre en resaltar neuronas y conexiones, y que la explicación textual evolucione sin mezclar responsabilidades.

Una primera utilidad implementada de la ventana aparece al pulsar una neurona. Al hacerlo se muestran los pesos que llegan a esa neurona y su sesgo asociado como se ve en la [Figura 6.19](#). Esta consulta manual permite inspeccionar el estado del modelo durante el entrenamiento, revisando los valores de las conexiones representadas. Si el usuario deselecciona la neurona, la ventana recupera la explicación anterior para no perder el contexto del paso que se estaba visualizando.

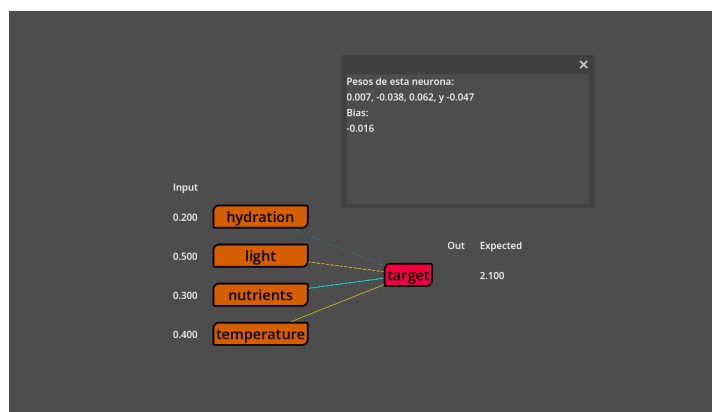


Figura 6.19: Entrenamiento de la red en la que se ve en la ventana los pesos y bias de la neurona resaltada.

Durante el forward pass, la ventana explica el cálculo de la neurona resaltada. Muestra las entradas recibidas, los pesos utilizados, el sesgo, la suma ponderada y la salida tras aplicar la función de activación. La explicación sigue la misma estructura matemática que usa el algoritmo y remite a la formulación general de la [Sección 5.2](#). En el caso de una neurona concreta, la ventana desarrolla esa operación con los valores de entrada y pesos correspondientes como se ve en la [Figura 6.20](#).

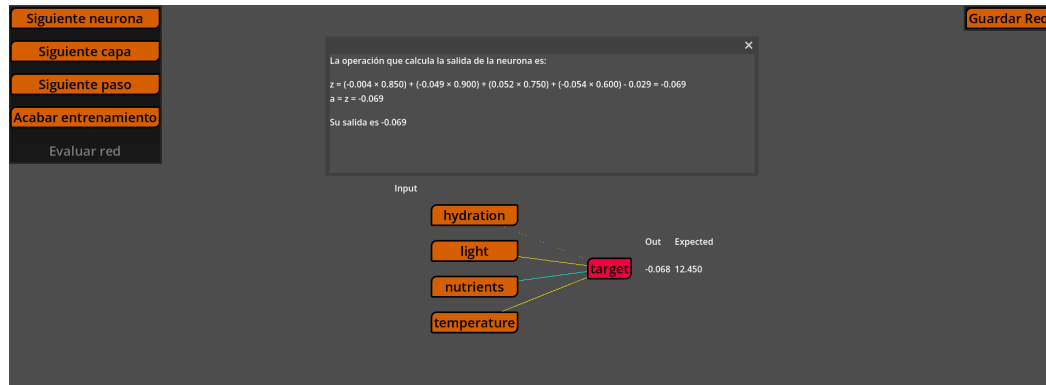


Figura 6.20: Entrenamiento de la red en la que se ve en la ventana una explicación en texto plano de la operación que se está calculando en la neurona resaltada.

En la fase de error, la ventana muestra la comparación entre salida y objetivo esperado. Indica el tipo de pérdida usado, el valor de la pérdida y los deltas que iniciarán la retropropagación. Esta explicación establece la transición entre el forward pass y el backward pass: primero se muestra qué ha producido la red y después se explica cómo ese resultado se transforma en una señal de corrección.

Durante el backward pass, la ventana adapta el texto al tipo de capa que se está procesando. En la salida puede mostrar la simplificación de softmax con entropía cruzada, o la combinación de derivada de pérdida y derivada de activación en otros casos. En las capas ocultas explica el gradiente entrante como la suma de contribuciones procedentes de la capa siguiente. Después muestra cómo ese delta genera gradientes de pesos y cómo esos gradientes modifican los parámetros.

Las fórmulas pueden mostrarse como texto plano o con un renderizado matemático basado en una petición [https a Math API \[17\]](https://mathjs.org/). Esta configuración era una opción booleana que se establecía desde código. El objetivo es mostrar con mejor calidad operaciones complicadas de representar en texto plano como las derivadas. En las Figuras 6.21 y 6.22 se ve la diferencia entre el texto plano y la fórmula renderizada.

Con la ventana informativa, la red combina la animación de valores con el razonamiento matemático que justifica cada cambio del modelo.

6.3.8 Inferencia con la red entrenada

Al terminar el entrenamiento, la aplicación habilita la evaluación de la red. Esta fase permite comprobar cómo se comporta el modelo entrenado con datos compatibles, usando el mismo tipo de visualización que se había preparado durante el aprendizaje. El cambio de modo marca el paso desde el ajuste de parámetros hacia la aplicación de los parámetros ya aprendidos.

La inferencia reutiliza el recorrido del forward pass. Cada muestra se carga, sus valores

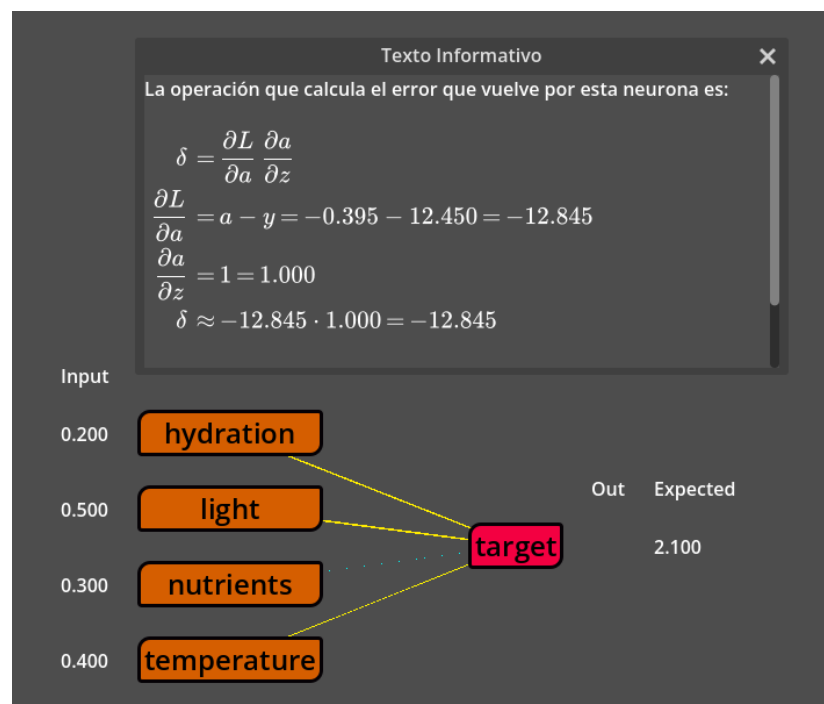


Figura 6.21: Fórmula de derivada renderizada en la ventana.

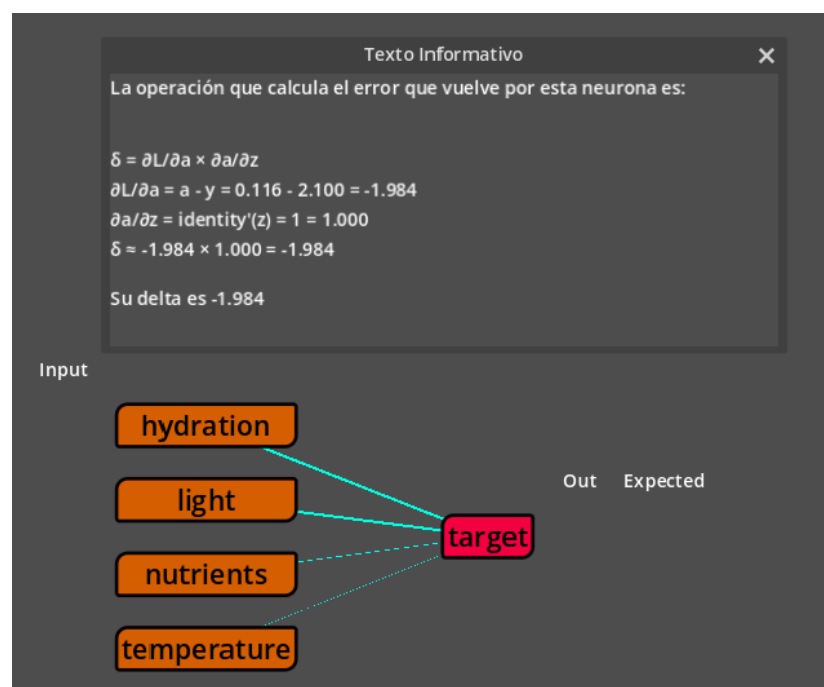


Figura 6.22: Fórmula de derivada en texto plano en la ventana

aparecen junto a la capa de entrada y después se propagan por las capas hasta producir una salida. La diferencia principal está en que el algoritmo se configura con tasa de aprendizaje nula y con el modo de inferencia activado. Cuando se completa la capa de salida, la muestra se da por finalizada: no se calcula retorno del error ni se actualizan pesos o sesgos.

Antes de ejecutar la inferencia, la aplicación valida el conjunto de datos elegido para evaluar. Debe tener los mismos atributos de entrada que el conjunto de datos usado en entrenamiento, las mismas salidas esperadas y el mismo orden de columnas. En la Figura 6.23 se puede ver como el panel de selección de dataset no permite avanzar a la inferencia con un dataset incompatible. Esta comprobación evita alimentar la red con vectores que tengan una estructura distinta a la que aprendió. Sin esa validación, una neurona de entrada podría recibir un atributo diferente al que representa su peso entrenado.

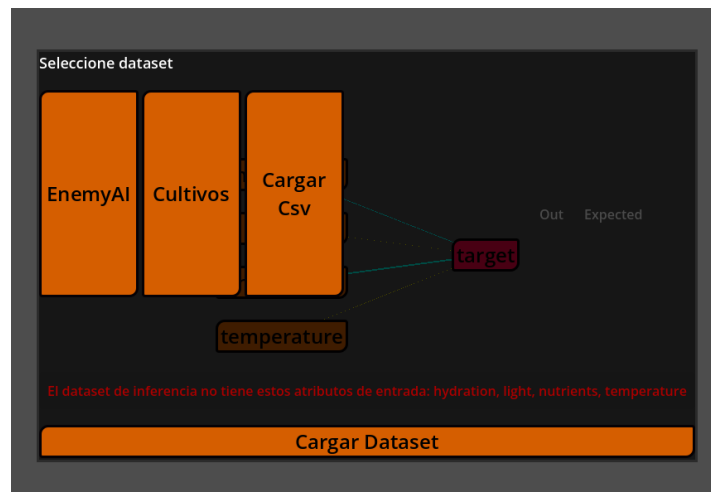


Figura 6.23: Panel de selección de dataset de inferencia en el que se ha intentado seleccionar el dataset *EnemyAI* para una red entrenada con *Cultivos*.

La visualización de datos se aprovecha también en este modo. Los valores de entrada vuelven a situarse junto a sus neuronas, se animan hacia la red y la columna de salida se completa a medida que avanza el cálculo. Al no haber backward pass, la evaluación se entiende como una lectura directa del modelo: se introduce una muestra y se observa qué salida produce.

Con esta fase queda completo el núcleo funcional de la red neuronal dentro de la aplicación. La herramienta permite escoger datos, crear una arquitectura coherente con ellos, entrenarla paso a paso, mostrar el forward pass, calcular y explicar el backward pass, actualizar los pesos de forma visible y evaluar la red entrenada mediante inferencia. Las mejoras posteriores se centran en pulir la interfaz general, adaptar la vista a distintos tamaños y mejorar la presentación visual, partiendo ya de un árbol y de una red funcionales y explicables.

La Figura ?? resume el flujo completo que queda implementado en la red neuronal. El

esquema se organiza en tres columnas dedicadas a la preparación, la máquina de estados y los elementos actualizados mediante Señales. Las flechas continuas representan el orden de ejecución y las discontinuas muestran las actualizaciones visuales o explicativas emitidas durante cada fase.

6.4 Mejoras de funcionalidades e interfaz de usuario: versión 1.0

Una vez completados los núcleos funcionales de los algoritmos el desarrollo cambia de enfoque. A partir de este punto, el trabajo se concentra en convertir ese conjunto de Escenas funcionales en una herramienta más coherente, preparada para la versión web y adecuada para una interacción continuada.

Esta fase se plantea como una revisión de tareas transversales que afectan a la experiencia completa, una vez terminada la incorporación de los algoritmos principales. Algunas mejoras afectan a toda la aplicación, como la navegación, el diseño, la adaptación a web o la limpieza del estado al cambiar de Escena. Otras se centran en reforzar partes concretas, como la evaluación del árbol o las explicaciones de la red neuronal. En conjunto, estas tareas permiten pasar de una demostración técnica de los modelos a una aplicación con una interfaz estructurada y con explicaciones integradas en el flujo de uso.

6.4.1 Entrada principal y navegación común

Una de las primeras tareas de pulido consiste en revisar el menú principal. Hasta ese momento, la pantalla inicial funcionaba solo como una forma rápida de entrar en cada algoritmo. Esto se cambió por un flujo de selección. El usuario escoge primero el algoritmo que quiere visualizar y, al hacerlo, la interfaz actualiza un panel descriptivo con información sobre esa opción. El botón de comienzo se habilita solo cuando existe una selección. De esta forma, el menú deja de actuar como una lista de botones y pasa a funcionar como punto de entrada explicativo al proyecto.

El panel descriptivo se acompaña de previsualizaciones de los algoritmos. Estas muestran una imagen representativa del algoritmo seleccionado. Esta decisión mantiene el criterio visual de la aplicación: siempre que es posible, la interfaz combina texto con una referencia gráfica relacionada con el contenido que se va a explorar.

Para planificar esta reorganización se preparó un boceto previo con Balsamiq Wireframes [18], una herramienta de creación de wireframes de baja fidelidad que permite definir la estructura de una interfaz antes de implementarla. El mockup de la Figura 6.24 sirvió como guía para ordenar la pantalla inicial alrededor de una selección de algoritmo, un panel descriptivo y una acción principal de comienzo. Durante la implementación final se ajustaron

tamaños e imágenes. La versión terminada, mostrada en las Figuras 6.25 y 6.26, conserva la organización general del boceto con cambios visuales derivados de su integración en Godot.

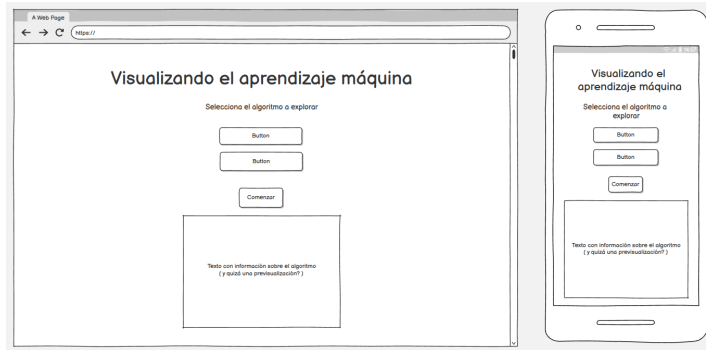


Figura 6.24: Mockup del menú principal realizado con Balsamiq antes de implementar la re-organización final de la entrada a la aplicación.

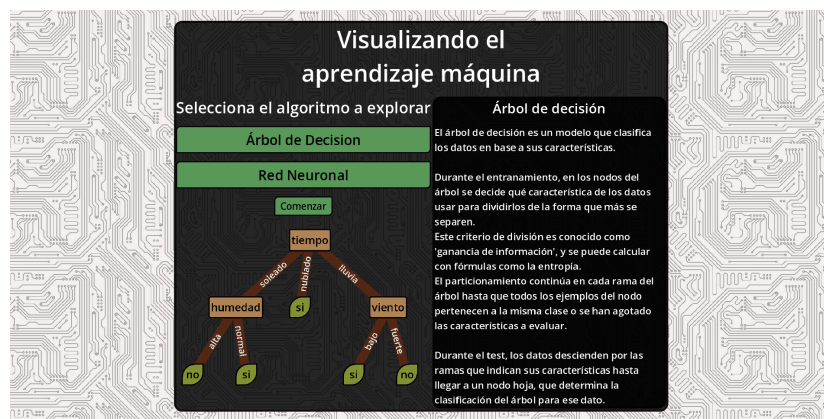


Figura 6.25: Versión final del menú principal, mostrando seleccionado el árbol de decisión.

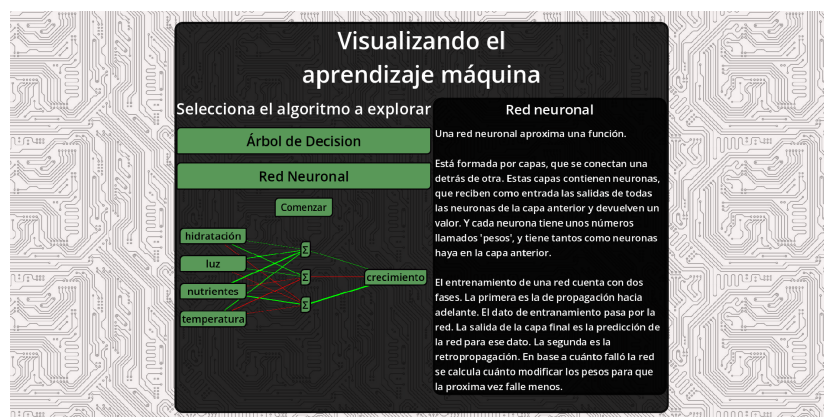


Figura 6.26: Versión final del menú principal, mostrando seleccionada la red neuronal.

La navegación interna se unifica mediante un menú común que se aprecia en la Figura 6.27. Tanto el árbol como la red incorporan una opción para volver al menú principal y, cuando corresponde, botones asociados a cargar o guardar el algoritmo que se esté explorando. Este menú se implementa como una *Escena* aparte con lógica autocontenida, de modo que las vistas no tienen que resolver por separado la navegación básica. En el árbol, los botones de carga y guardado abren el explorador de archivos local para cargar en la aplicación o guardar en el ordenador respectivamente la estructura entrenada. En la red, el de guardado abre el panel de exportación del modelo. Para la red la carga se hace desde el menú de edición de la arquitectura. Al volver al menú principal también se limpia el estado compartido de la red neuronal. Esta limpieza evita que una arquitectura creada en una sesión anterior interfiera en la construcción de una nueva red. Con esta mejora, la aplicación puede reutilizarse dentro de la misma sesión web sin reiniciar la página.

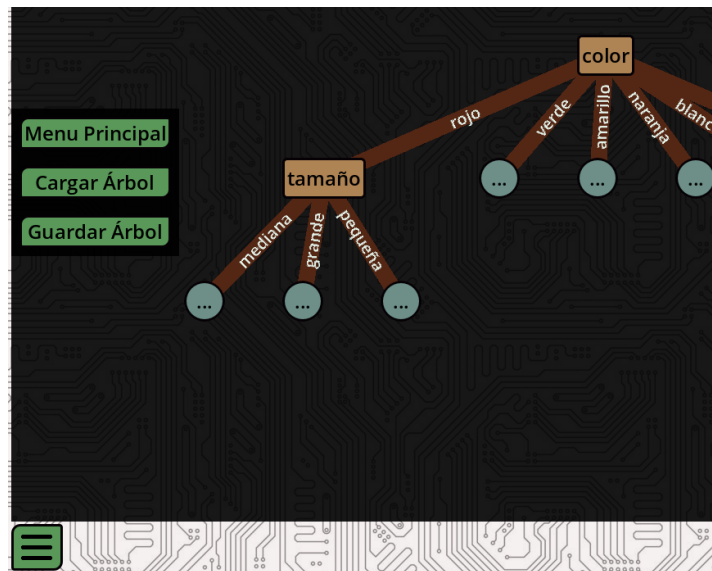


Figura 6.27: Menú de navegación que permite volver al menú principal y guardar o cargar estructuras de los algoritmos.

6.4.2 Adaptación a pantallas de distintos tamaños y orientaciones

Otra tarea central de esta fase es la adaptación de la interfaz a tamaños de pantalla distintos. La aplicación se publica como página web, por lo que no puede asumir una única resolución de escritorio. El mismo contenido debe seguir siendo utilizable en una ventana amplia, en una pantalla vertical y en dispositivos con interacción táctil.

Antes de llevar esta adaptación a Godot, se definieron con Balsamiq dos disposiciones base para las vistas de los algoritmos. Los bocetos de la Figura 6.28 fijan la diferencia entre la versión horizontal, pensada para colocar la visualización y la ventana informativa en paralelo,

y la versión vertical, pensada para apilar los mismos elementos en pantallas estrechas. Esta planificación reduce la dependencia de posiciones absolutas y permite tratar la adaptación como una decisión de estructura desde el inicio, más allá de un ajuste de escala.

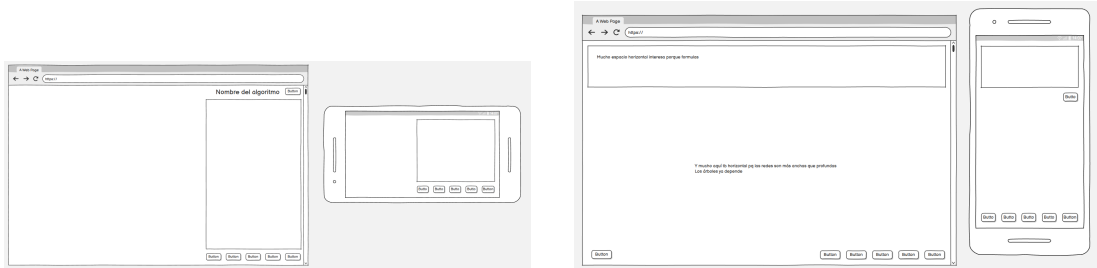


Figura 6.28: Mockups de las disposiciones horizontal y vertical de las vistas de algoritmo realizados con Balsamiq.

Bajo esta premisa el menú principal se reorganiza alrededor de contenedores de interfaz de Godot. Se incorporan márgenes, zonas centradas, paneles con anchura máxima y contenedores desplazables. Con esta estructura, el contenido puede reducir su anchura y activar desplazamiento vertical de forma automática cuando la pantalla es pequeña. Los tamaños de fuente, los márgenes y las dimensiones mínimas de los botones se ajustan para conservar legibilidad sin depender de posiciones absolutas. Los resultados ya se mostraron en la Sección 6.4.1. También se puede ver el resultado en móvil en la Figura 6.29.

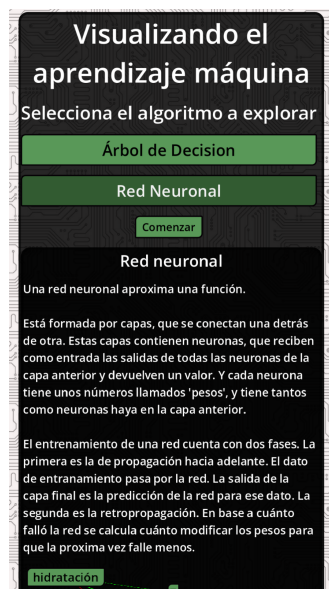


Figura 6.29: Vista del menu principal en un movil.

Tanto el árbol de decisión como la red neuronal se adaptan a esta lógica. A partir de esta

revisión, la vista se divide en variantes vertical y horizontal, y el menú principal decide cuál cargar según la proporción de la ventana. La ventana deja de comportarse como un elemento flotante y queda integrada y fija en la *Escena*. En pantallas anchas, la vista del algoritmo y la ventana informativa pueden situarse en paralelo, aprovechando mejor el espacio horizontal. En pantallas verticales, los elementos se colocan en una estructura apilada, con desplazamiento cuando el contenido excede el área visible. Esta organización adapta los paneles principales al tamaño disponible en el navegador.

Durante la creación de la red también se separa el espacio de edición del espacio de visualización. En la versión anterior, el panel de edición de la red se situaba sobre la propia red durante la configuración, como se aprecia en la Figura 6.30. En la versión revisada, un contenedor agrupa el menú de creación y el panel de conjunto de datos, mientras que otro mantiene la red preliminar siempre visible, como se muestra en la Figura 6.31. Cuando comienza el entrenamiento o la inferencia, los paneles de creación se ocultan para dar prioridad a los controles del algoritmo y a la ventana informativa.



Figura 6.30: Creación de la red con el menú traslúcido situado sobre la visualización.

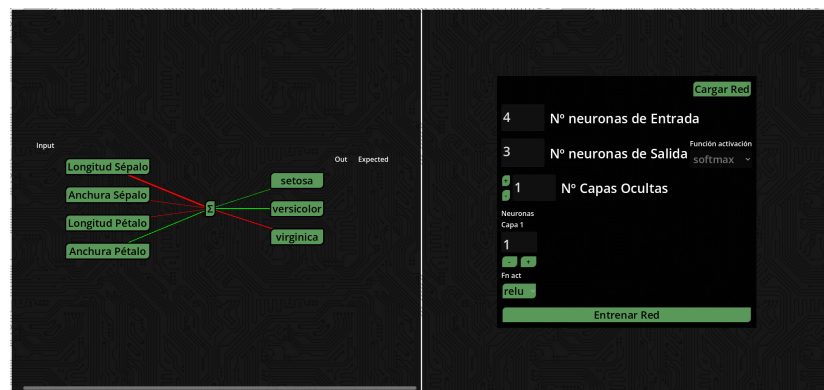


Figura 6.31: Creación de la red con el menú de creación situado al lado de la visualización.

Para las zonas grandes, como el árbol entrenado o la red con varias capas, se mejora el desplazamiento y el zoom. El contenedor desplazable incorpora control con rueda, combinaciones de teclado y gestos táctiles. Con un dedo se puede arrastrar la vista, y con dos dedos se puede aplicar zoom mediante pinza. Estas interacciones se aplican al árbol, donde algunas ramas pueden alejarse del centro, y a la red, donde varias capas densas pueden ocupar una superficie amplia.

Los últimos cambios de adaptación a pantalla consisten en reducir márgenes y separaciones entre los contenidos. En pantallas pequeñas, esos espacios ocupan una parte significativa del área útil. Por ello se recortan de forma parcial, manteniendo la organización de los paneles y aumentando el espacio disponible para el contenido interactivo.

6.4.3 Identidad visual y legibilidad de la interfaz

Se revisa para este punto la estética general de la aplicación. El objetivo es unificar la identidad visual y hacer que los elementos principales se distingan con claridad. Se actualizan colores de fondo, estilos de botones, bordes, textos y temas de paneles. El fondo pasa a usar una textura inspirada en una placa base.

También se actualiza el redibujado de conexiones. Al cambiar la arquitectura de la red se ejecutaban la animación de aparición de todas las conexiones. Ahora solo las conexiones nuevas se animan. Con ello, la previsualización mantiene mayor estabilidad cuando se modifica el número de capas o neuronas.

En el árbol de decisión se ajusta el grosor de las conexiones y el contraste de las etiquetas de rama. Las conexiones y los **nodos** de decisión adoptan colores cercanos al marrón, mientras que las hojas mantienen el verde, inspirando los colores de una planta.

En la red neuronal se revisa el color y grosor de las conexiones para representar mejor los pesos. Como se muestra en la Figura 6.32, las conexiones se coloreaban inicialmente con un gradiente basado en el peso de la conexión. Las conexiones con peso positivo partían de un color cian para valores bajos y cambiaban a verde según aumentaba el peso. Las conexiones con peso negativo partían de amarillo y cambiaban a rojo cuanto más negativo era el valor. Esta codificación se sustituye por una representación más directa: los pesos positivos se muestran en verde, los negativos en rojo y los valores cercanos a cero en blanco. La magnitud se sigue comunicando mediante el grosor, con límites mínimos y máximos para evitar líneas inapreciables o desproporcionadamente grandes. En la Figura 6.33 se puede ver la representación final.

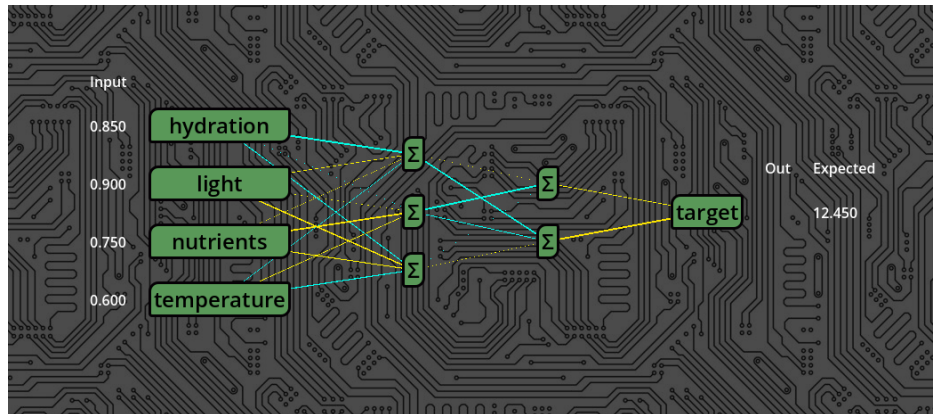


Figura 6.32: Red neuronal con conexiones sin limite de grosor coloreadas mediante gradientes desde el cian hasta el verde para valores positivos y desde el amarillo hasta el rojo para valores negativos.

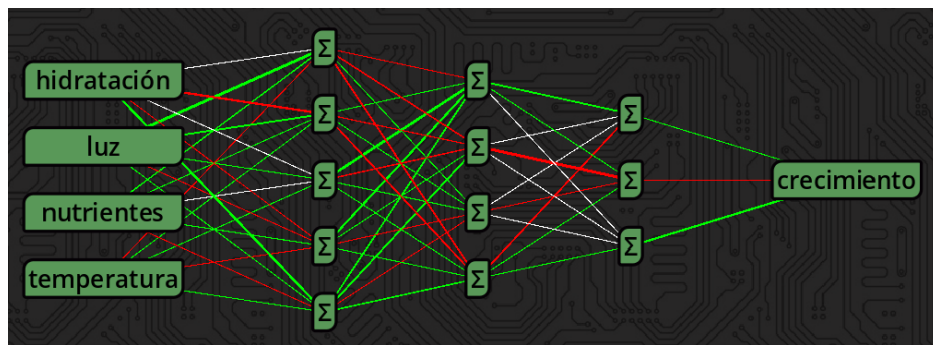
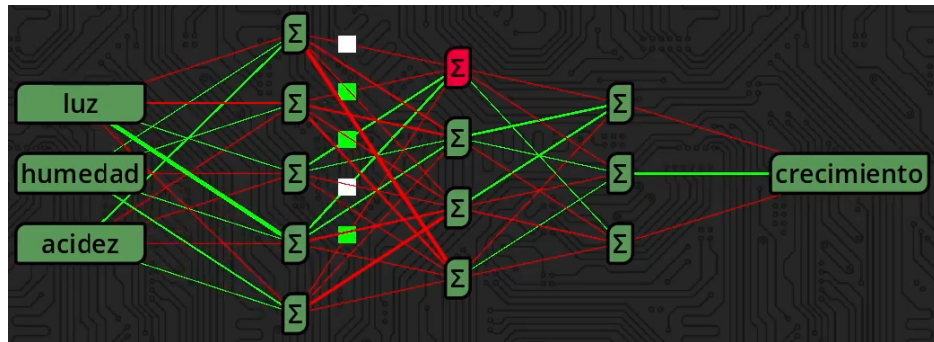


Figura 6.33: Red neuronal con conexiones con límite de grosor coloreadas en verde para valores positivos, en rojo para valores negativos, y en blanco los más cercanos a 0.

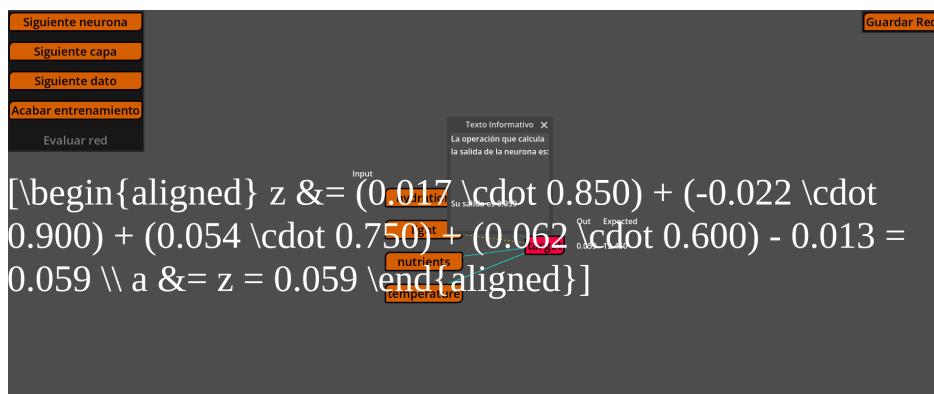
Y tanto durante la propagación como la retropropagación de la red se añade un indicador visual más explícito para las actualizaciones de conexiones. Las conexiones ya contaban con animación, pero los cambios de baja magnitud podían pasar desapercibidos. La solución consiste en añadir una figura que retrocede por la conexión actualizada y cuyo color indica el signo de la actualización. De esta forma se representa que las conexiones se modifican durante el backward pass, incluso cuando la variación del grosor es pequeña. En la Figura 6.34 se puede apreciar esta visualización en estático.

Figura 6.34: Red en medio de la animación de *forward pass*.

6.4.4 Renderizado de fórmulas matemáticas

La ventana informativa de la red y la del árbol necesitan mostrar expresiones matemáticas para completar las explicaciones. En las primeras versiones, las fórmulas podían representarse como texto plano. Esa solución resultaba limitada para explicar operaciones como la derivada de los pesos, o expresiones matriciales. También se probó a renderizar las fórmulas por medio de una petición https a Math API, un servicio que generaba la imagen a partir de una expresión LaTeX. Pero esto introducía una latencia de segundos entre que aparecía la explicación y se mostraba la fórmula. Por ello se dedica una tarea específica a integrar un sistema de renderizado LaTeX dentro de la aplicación.

Inicialmente se probó una solución basada en MathJax y elementos HTML superpuestos al lienzo de Godot. Sin embargo el resultado siempre acababa con el texto de latex en crudo ocupando toda la pantalla como se ve en la Figura 6.35. Esto ocurre porque, como se trató en la Sección 6.1.2, el sistema de exportación de Godot se fundamenta en un elemento *canvas*, en el cual MathJax no puede renderizar de forma nativa.

Figura 6.35: Error en el que MathJax no se renderiza adecuadamente en el *canvas* de Godot

La solución final se desarrolla como una extensión propia del motor, programada en Rust.

El renderizador convierte expresiones LaTeX en SVG usando la librería Ratex [19] y las presenta como texturas dentro de Godot. El componente mantiene caché de fórmulas según texto, color, escala, tamaño de fuente y padding, y emite Señales cuando el renderizado se completa o falla. También se prepara su compilación para web mediante WebAssembly, de manera que las fórmulas funcionen en la versión exportada a HTML5.

Una vez integrado, el renderizador se incorpora a la ventana de la red neuronal y a la del árbol de decisión. En la red se usa para acompañar las explicaciones del forward pass, la pérdida, los deltas y la actualización de pesos. En el árbol se usa para mostrar la fórmula de la entropía en los pasos donde el algoritmo compara atributos. Cuando la ventana muestra otros contenidos, como selección de **nodos** o evaluación de datos, la fórmula se oculta para no mantener información fuera de contexto.

El ajuste de escalado requirió varias iteraciones. Se revisaron tamaños de fuente, padding, modo de estirado, tamaño mínimo lógico y normalización de SVG para que la fórmula tenga una dimensión natural dentro del panel. Este trabajo afecta especialmente a la versión web, donde un escalado incorrecto puede producir texturas borrosas o demasiado grandes.

6.4.5 Explicaciones de mayor nivel en la red neuronal

La red neuronal ya explicaba cálculos neurona a neurona, pero esta fase amplía la granularidad de las explicaciones aprovechando la posibilidad de renderizar formulas de la Sección 6.4.4. El panel de entrenamiento permite avanzar por neurona, por capa, por muestra o completar el entrenamiento. La ventana informativa debe adaptarse a esos niveles para que cada modo de avance tenga una explicación adecuada.

Para el forward pass se añaden explicaciones de capa completa. El algoritmo guarda la información del último avance de capa y la ventana puede mostrar la operación agregada como se muestra en la Figura 6.36. Esto conecta la visualización detallada de neuronas individuales con la formulación matricial habitual de las redes densas.

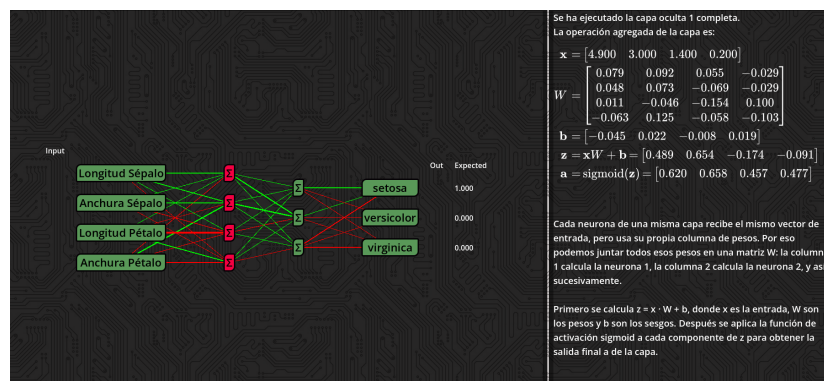


Figura 6.36: Texto y fórmulas de la propagación de la red por capas.

El backward pass recibe un tratamiento similar. En la capa de salida se muestra la pérdida usada y el delta inicial. En capas ocultas se explica cómo llega el gradiente desde la capa siguiente y cómo se transforma en deltas propios. En la Figura 6.37 se puede ver el texto explicativo.

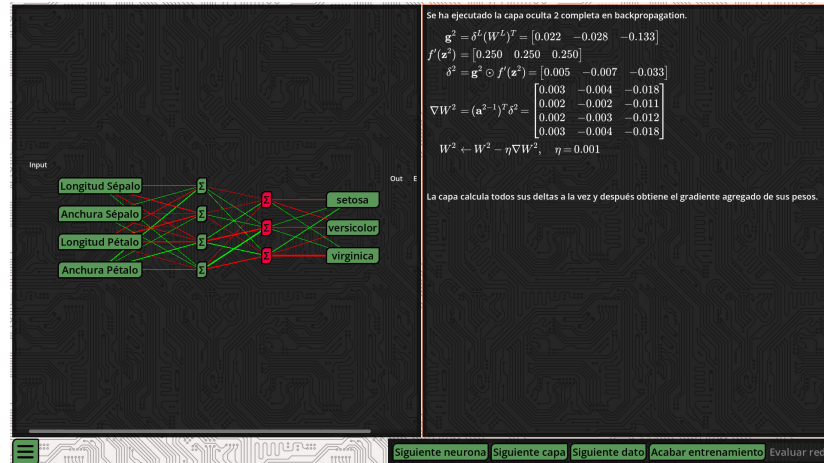


Figura 6.37: Texto y fórmulas de la retropropagación de la red por capas.

También se añade una explicación al pasar todo el proceso para una muestra. Cuando el usuario avanza un dato entero, la ventana resume las entradas utilizadas, la salida producida, el objetivo esperado y el valor de pérdida como se ve en la Figura 6.38. Este modo resulta útil para observar el efecto global de una muestra sin detenerse en cada neurona.

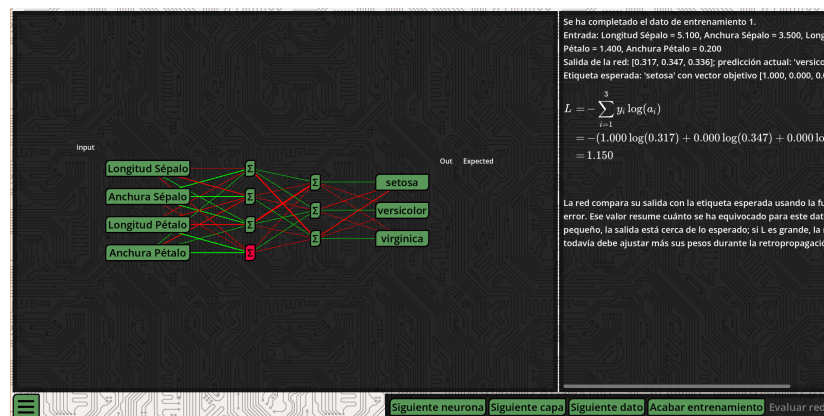


Figura 6.38: Texto y fórmulas al entrenar con dato en un paso.

Este cambio refuerza la idea de que el entrenamiento puede mirarse con distintos niveles de detalle. Un usuario puede estudiar una neurona concreta, revisar una capa como operación matricial o seguir una muestra completa como unidad de aprendizaje.

6.4.6 Gráficas de funciones de activación e inspección de neuronas y nodos

Se incorpora el poder consultar la información interna de las neuronas de la red y **nodos** del árbol. Antes, la información de la pureza de un nodo del árbol se mostraba al arrastrar el ratón por encima como un número, sin acompañar de ninguna explicación. Ahora, al pulsar uno de ellos, la ventana explicativa cambia su contenido para mostrar con más contexto la misma información como se aprecia en la Figura 6.39. Lo mismo sucede en las neuronas. A mayores, el elemento seleccionado queda resaltado para indicar a qué parte de la estructura se refiere la información. Una nueva pulsación restaura la explicación anterior. El avance del algoritmo puede continuar después de la consulta, y también es posible inspeccionar otros **nodos** o neuronas sin reiniciar el proceso activo.

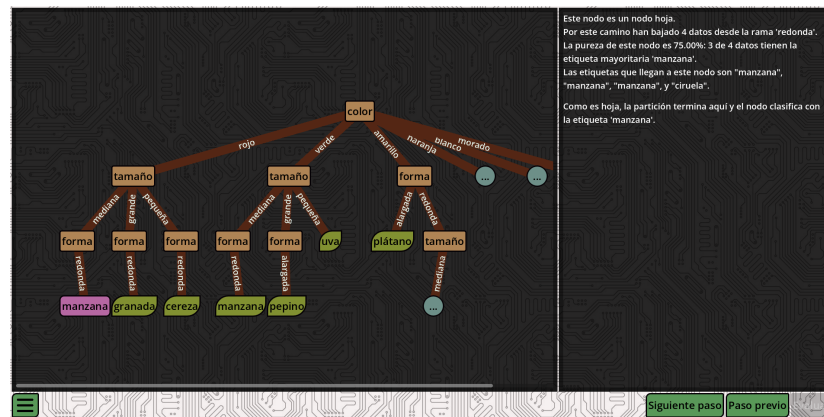


Figura 6.39: Ejemplo de un nodo resaltado durante el entrenamiento de un árbol de decisión.

La explicación de las funciones de activación se refuerza con gráficas. Se generan imágenes para funciones como ReLU, sigmoide e identidad. Las gráficas se preparan con fondo transparente, ejes claros y cuadrícula tenue para integrarse en la estética oscura de la aplicación. Después se incorporan como texturas a la ventana informativa de la red. Durante el forward pass, la ventana puede mostrar la gráfica asociada a la función de activación de la capa que se está calculando como se ve en la Figura 6.40. Cuando la explicación pasa a otro tipo de contenido, como la carga de datos, la retropropagación o el fin del entrenamiento, la gráfica se limpia junto con la fórmula anterior. Esta limpieza mantiene la ventana centrada en el paso actual y evita que una representación matemática permanezca visible fuera de contexto.

El último paso consiste en marcar sobre la gráfica el punto evaluado por la neurona. La ventana toma la entrada ponderada, la salida activada y la función correspondiente, y transforma esos valores en una posición dentro del área útil de la imagen. Sobre ella se dibuja un marcador rojo que indica qué punto de la curva se está usando en ese cálculo concreto, como también se aprecia en la Figura 6.40. Así se conectan el valor numérico de entrada, la fórmula aplicada y la forma de la función de activación dentro de la ventana explicativa.



Figura 6.40: Etapa del *forward pass* en la que se ve en la ventana la función de activación dibujada junto con el punto que marca la salida de la función.

6.4.7 Selección de conjuntos de datos

La carga de datos ya existía en fases anteriores. En esta etapa se mejora la usabilidad del panel de carga al mejorarlo a nivel estético y de funcionalidades. Visualmente se aprecia en la Figura 6.41 mejoras en la legibilidad gracias a la nueva distribución de la información en el panel. Y en cuanto a funcionalidades, se implementó, para su uso desde el navegador, una conexión con JavaScript que permite leer archivos desde la aplicación exportada al navegador. Esta se usa cuándo la aplicación detecta que el entorno de ejecución es web. Se encarga de llamar al recurso del navegador que permite el acceso a archivos en local, para que después el motor de Godot se encargue de procesar lo seleccionado. Hasta que se implementó este cambio, en la versión web se habría un menú nativo de Godot sin acceso al ordenador del usuario. Las pruebas de carga se podían seguir realizando, pero desde el editor. Con esta funcionalidad implementada se puede también cargar con normalidad un csv también desde la aplicación desplegada. Esto sería usado también para la carga y cuardado de modelos, como se detalla en la Sección 6.4.9.

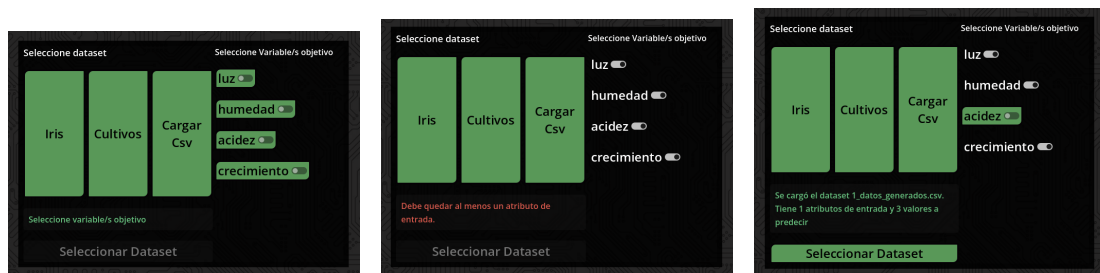


Figura 6.41: Diferentes estados de la carga de un dataset desde un csv.

En el árbol de decisión se amplían los conjuntos de datos internos. El conjunto de frutas incorpora más instancias, clases con características compartidas y casos que pueden generar

hojas impuras. También se añade el conjunto de tenis, un ejemplo clásico de árboles de decisión con atributos categóricos. Estos conjuntos permiten observar estructuras más variadas que las usadas en las primeras pruebas. También se incorpora al árbol la selección explícita de la variable objetivo, una función que ya estaba presente en el selector de datos de la red.

En la red neuronal se revisan también los conjuntos internos. El conjunto de prueba inicial se sustituye por *Iris* [20], un ejemplo clásico de clasificación multiclase con cuatro entradas numéricas y salida *softmax*, y se aumentan las entradas de *Cultivos*, un dataset de regresión sintético.

También se añaden validaciones básicas de estructura. La aplicación informa si el archivo no puede abrirse, si no contiene suficientes columnas, si no hay filas válidas o si todavía no se ha escogido una variable objetivo. El botón de selección permanece desactivado hasta que la configuración es suficiente para continuar. Así se evita iniciar el algoritmo con datos incompletos.

Por último se traducen todos los conjuntos de datos al castellano. Los nombres de atributos y valores pasan a estar alineados con el idioma de la interfaz y de la memoria. Esto evita mezclar explicaciones en castellano con ejemplos en inglés y mejora la coherencia de los textos generados por la ventana.

6.4.8 Evaluación visual del árbol de decisión

La evaluación del árbol se revisa para completar la representación del recorrido de los datos por la estructura entrenada. Ya existía una primera versión en la que los datos recorrían el árbol hasta una hoja. En esta fase se mejora la representación visual de cada ejemplo y la explicación asociada a cada decisión.

Los datos dejan de representarse como cuadrados planos y pasan a mostrarse mediante botones, implementados como **Nodos** de interfaz con una estética propia. Cuando se crean, reciben un número único derivado de su etiqueta mediante una función hash. Ese valor determina características visuales como el color o la forma. De esta manera, todos los datos a evaluar que comparten etiqueta mantienen el mismo aspecto. Cuando una de las características del dato es el color, como ocurre en el conjunto de frutas, el color del elemento visual se aproxima al valor indicado por ese atributo.

Cuando varios datos llegan a la misma hoja, se apilan debajo de ella. Esta solución evita superposiciones y permite ver cuántos ejemplos han terminado en una clasificación concreta. Si la pila queda fuera del área inicial del árbol, el lienzo se amplía para que siga siendo accesible mediante desplazamiento.

Otra mejora introducida es el avance **nodo** a **nodo**. En lugar de mover un dato desde la raíz hasta la hoja en una única animación, la aplicación mantiene un dato activo y un **nodo** activo. Cada pulsación avanza una decisión, empezando cuando el dato entra por la raíz, después

cuando se escoge la rama correspondiente al valor del atributo, y así sucesivamente hasta que llega a una hoja. Este cambio aplica a la evaluación la misma lógica de avance incremental utilizada durante la construcción del árbol.

Para dar robustez a las animaciones, cada dato mantiene su propia cola de movimientos. Así, si se encadenan desplazamientos o se pulsa de nuevo mientras una animación está en curso, el flujo no depende de un único estado global. Cada elemento visual sabe qué movimientos tiene pendientes y los ejecuta en orden.

La ventana informativa acompaña el recorrido. Al seleccionar un dato, muestra su etiqueta real y sus atributos. Al avanzar por el árbol, explica si el dato acaba de entrar por la raíz, si está pasando por un **nodo** interno o si ha llegado a una hoja. En la hoja compara la etiqueta real con la clase asignada por el árbol. Si coinciden, indica que la clasificación ha sido correcta. Si no coinciden, distingue entre una hoja pura y una hoja impura: en el primer caso, el entrenamiento no contenía ejemplos de esa etiqueta en esa región; en el segundo, la etiqueta real no fue la mayoritaria entre los ejemplos que llegaron a esa hoja. Si se pulsa uno de los datos en evaluación, la ventana muestra su información interna, formada por sus atributos y su etiqueta. La inspección del árbol ya construido se mantiene disponible durante la evaluación, de forma que los **nodos** del árbol pueden seguir seleccionándose para consultar su información.

En la Figura 6.42 se puede ver el resultado.

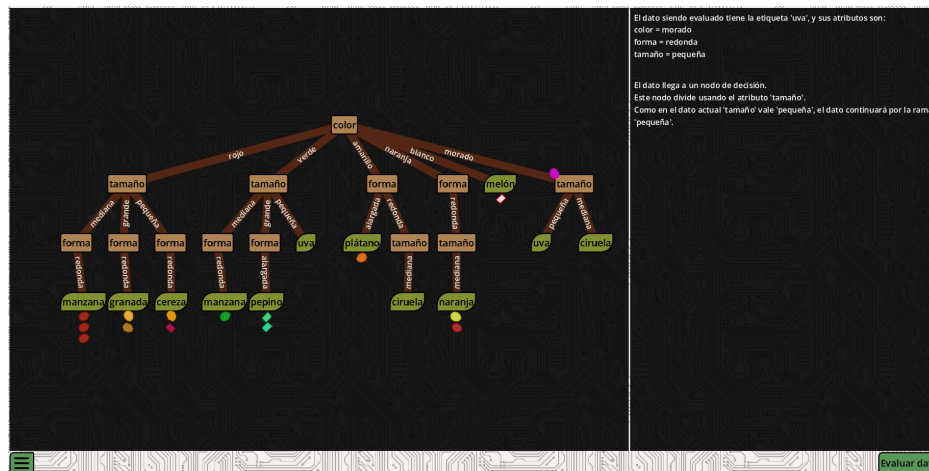


Figura 6.42: Árbol de decisión durante el proceso de evaluación del conjunto de frutas.

6.4.9 Persistencia, carga y validación de modelos

La última tarea se centra en conservar resultados, recuperarlos desde archivo y contrastar los cálculos que muestra la aplicación. Esta necesidad aparece en los dos modelos y se resuelve con formatos distintos. En el árbol de decisión se guarda una estructura visual y lógica propia de la aplicación. En la red neuronal se guardan los parámetros de un modelo entrenado en un

formato que pueda usarse también fuera de la aplicación.

En la versión web, tanto la carga como el guardado de archivos necesitan una integración específica con el navegador. Para ello se utiliza una conexión con JavaScript que permite leer archivos elegidos por el usuario y generar descargas desde la aplicación exportada. Esta conexión tiene un papel transversal, ya que se usa tanto para guardar y cargar ambos tipos de modelos, como para cargar csvs como se cuenta en la Sección 6.4.9

En el árbol de decisión se implementa guardado y carga en formato JSON. El guardado conserva la estructura del árbol, la información visible de sus **nodos** y ramas, y los metadatos del conjunto de datos cuando están disponibles. Esto permite recuperar un árbol con su forma visual y con la información necesaria para seguir inspeccionándolo o evaluándolo. Para la carga, primero se valida la estructura, después se vacía el árbol que esté en la aplicación en ese momento y vuelve por último vuelve a crear los **nodos** en base a lo que indique el archivo. Tras una carga válida, el controlador configura el árbol como una estructura ya entrenada, preparando la **Escena** para ello.

En la red neuronal, el guardado de la red se sitúa en el menú de navegación, como el árbol. Mientras, la carga de modelos se sitúa en el menú de creación, ya que ese menú define o sustituye la arquitectura antes de entrenar o evaluar. La aplicación trabaja con ONNX [21] como formato implementado para esta operación. ONNX, siglas de *Open Neural Network Exchange*, es un formato abierto para representar modelos de aprendizaje automático y facilitar su intercambio entre herramientas compatibles. Tanto los modelos importados como exportados se procesan en un **Nodo** dedicado, que convierte el archivo a las estructuras internas de la aplicación o viceversa. Si la carga falla, el menú muestra un mensaje de error, si se completa, la red visible se reconstruye con el modelo importado.

Antes de confirmar que una red cargada puede usarse, la aplicación valida su compatibilidad con el conjunto de datos seleccionado. El modelo debe tener el número esperado de neuronas de entrada y de salida. Cuando alguna dimensión no coincide, la interfaz informa de la diferencia entre la arquitectura cargada y la requerida por los datos actuales. Esta comprobación impide entrenar o ejecutar inferencia con un modelo que no puede recibir las columnas seleccionadas o que no produce las salidas esperadas.

La implementación de esta exportación fué validada con un script de Python. En este se verificaba que la red de la Figura 6.43, extraída de la aplicación, era reconocida por la librería de ONNX y que sus parámetros coincidían. En la neurona de salida se observan pesos cercanos a 1.943, 1.929 y -0.965 , junto con un sesgo cercano a 0.994.

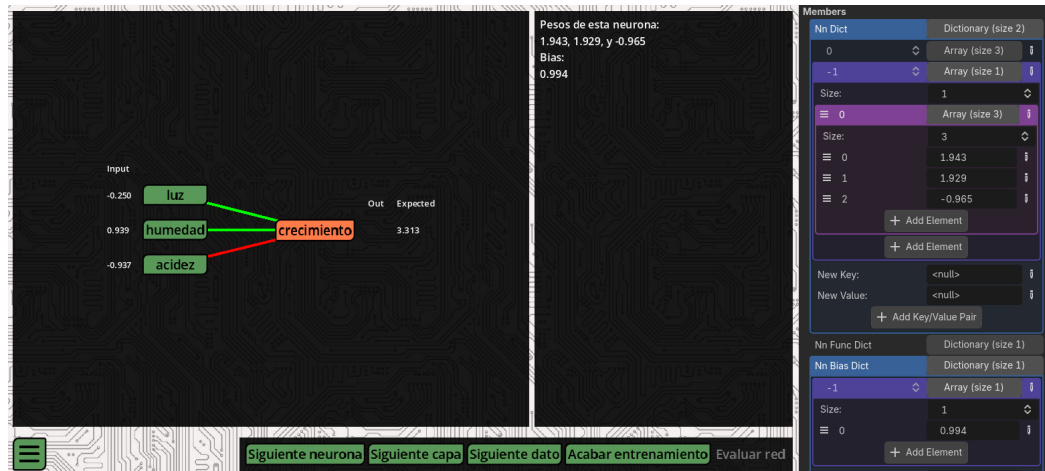


Figura 6.43: Red neuronal entrenada dentro de la aplicación, con los pesos y el sesgo de la neurona de salida visibles en la interfaz.

Como se muestra en la Figura 6.44, los valores leídos desde el archivo coinciden con los mostrados por la aplicación salvo por el redondeo hecho por la interfaz del editor de Godot. Con esta comparación se comprueba que la exportación conserva el orden y el valor de los parámetros aprendidos.

```
Éxito: El archivo 'network.onnx' se ha importado y verificado correctamente.
W_out:
[[ 1.9381292  1.9237686 -0.96262616]]
B_out:
[0.99369425]
```

Figura 6.44: Lectura externa del archivo ONNX exportado desde la aplicación, con los pesos y el sesgo de la red entrenada.

También se comprobó el sentido inverso. Para ello se generó fuera de la aplicación una red sintética con pesos exactos 2.0, 2.0 y -1.0 , y con sesgo 1.0. La salida de terminal de la Figura 6.45 recoge esos valores antes de importar el modelo en Godot.

```
Pesos finales del modelo:
tensor([[ 2.0000,  2.0000, -1.0000]])
Bias final del modelo:
tensor([1.0000])
```

Figura 6.45: Red sintética exportada a ONNX desde un entorno externo, con pesos y sesgo conocidos.

Al importar ese archivo en la aplicación, la red reconstruida conserva los mismos pa-

rámetros, tal como se muestra en la Figura 6.46. Esta prueba completa la comprobación de interoperabilidad. Los modelos entrenados en la aplicación pueden exportarse a ONNX y los modelos creados fuera pueden importarse con sus pesos y sesgos correctamente asignados.

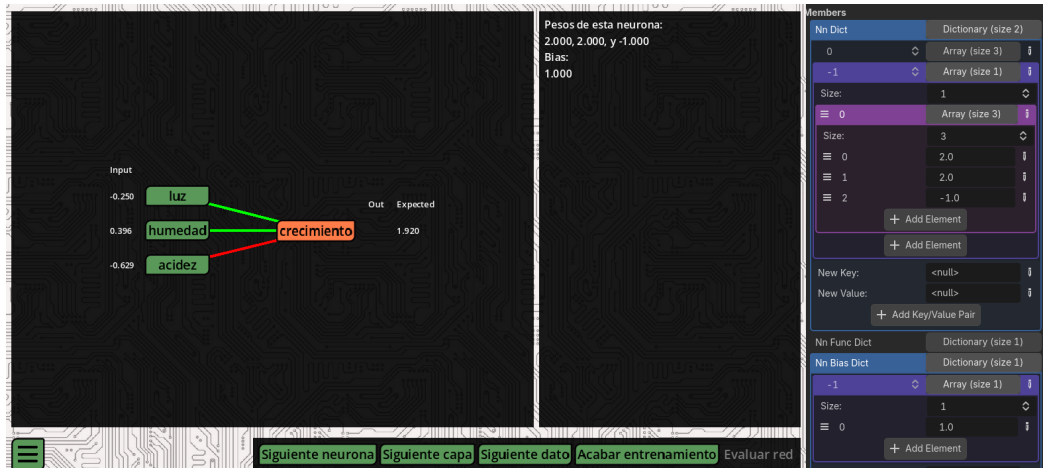


Figura 6.46: Red sintética importada en la aplicación desde ONNX, con los pesos y el sesgo esperados visibles en la interfaz.

6.4.10 Comprobación de resultados con bibliotecas externas

Junto con la persistencia de los modelos, se contrastaron los resultados de los algoritmos implementados con librerías de referencia. El objetivo es verificar que la aplicación representa una ejecución didáctica cuyos cálculos coinciden con los obtenidos mediante bibliotecas estándar. Concretamente scikit-learn [22] para el árbol de decisión y PyTorch [23] para la red neuronal.

En el caso del árbol de decisión, la comprobación se realizó con un conjunto categórico sintético. Primero se entrenó una referencia externa con scikit-learn usando los mismos datos. La Figura 6.47 muestra la estructura obtenida desde el script de comprobación, impresa en la terminal. La captura recoge solo los primeros **nodos** del árbol completo por limitación de espacio. Esa parte permite identificar la raíz, las primeras ramas y varias hojas resultantes.

Al entrenar el árbol con ese mismo conjunto de datos desde la aplicación se obtiene la estructura de la Figura 6.48. La coincidencia entre ambas salidas se aprecia en la raíz, en las ramas principales y en las etiquetas finales asociadas a las hojas. Esta comparación confirma que la implementación del algoritmo en Godot toma las mismas decisiones de división que la referencia externa para este caso de prueba, y que la representación visual mantiene la información lógica necesaria para interpretar el modelo entrenado.

```

[Nodo 0] color (pureza: 4/22 = 0.1818)
|-- amarillo
| [Nodo 1] forma (pureza: 1/3 = 0.3333)
| |-- alargada
| | [Nodo 2] plátano (pureza: 1/1 = 1.0000)
| |-- redonda
| | [Nodo 3] tamaño (pureza: 1/2 = 0.5000)
| | |-- mediana
| | | [Nodo 4] ciruela (pureza: 1/2 = 0.5000)
|-- blanco
| [Nodo 5] melón (pureza: 1/1 = 1.0000)
|-- morado
| [Nodo 6] tamaño (pureza: 2/3 = 0.6667)
| |-- mediana
| | [Nodo 7] ciruela (pureza: 2/2 = 1.0000)
| |-- pequeña
| | [Nodo 8] uva (pureza: 1/1 = 1.0000)
|-- naranja
| [Nodo 9] forma (pureza: 1/2 = 0.5000)
| |-- redonda
| | [Nodo 10] tamaño (pureza: 1/2 = 0.5000)
| | |-- mediana
| | | [Nodo 11] naranja (pureza: 1/2 = 0.5000)
|-- rojo
| [Nodo 12] tamaño (pureza: 3/8 = 0.3750)
| |-- grande
| | [Nodo 13] forma (pureza: 1/2 = 0.5000)
| | |-- redonda
| | | [Nodo 14] granada (pureza: 1/2 = 0.5000)
| |-- mediana
| | [Nodo 15] forma (pureza: 3/4 = 0.7500)
| | |-- redonda
| | | [Nodo 16] manzana (pureza: 3/4 = 0.7500)
| |-- pequeña
| | [Nodo 17] forma (pureza: 1/2 = 0.5000)
| | |-- redonda
| | | [Nodo 18] cereza (pureza: 1/2 = 0.5000)
|-- verde
| [Nodo 19] tamaño (pureza: 1/5 = 0.2000)
| |-- grande
| | [Nodo 20] forma (pureza: 1/2 = 0.5000)
| | |-- alargada
| | | [Nodo 21] pepino (pureza: 1/2 = 0.5000)
| |-- mediana

```

Figura 6.47: Primeros **nodos** de la estructura final de un árbol de decisión obtenido con scikit-learn a partir de un conjunto sintético.

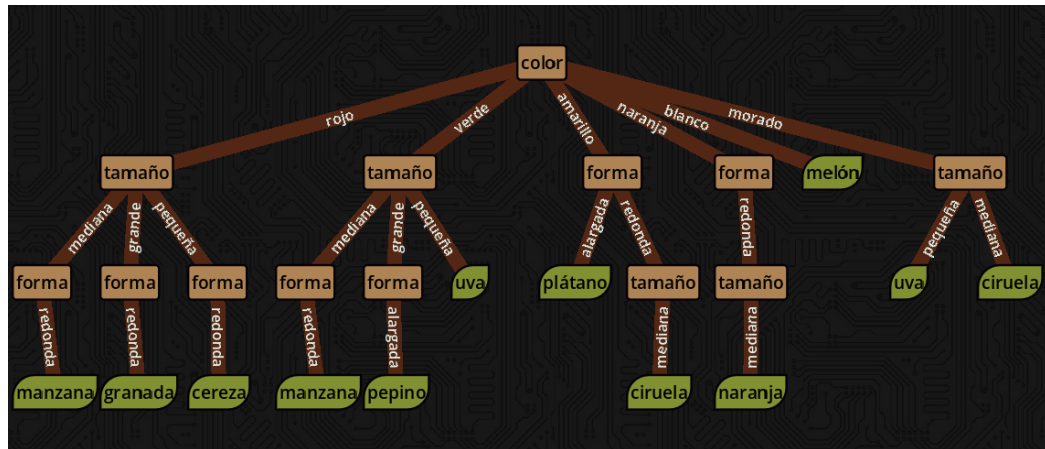


Figura 6.48: Árbol de decisión generado por la aplicación con el mismo conjunto sintético usado en la referencia externa.

Para la red neuronal se usa un conjunto sintético y una red equivalente implementada en PyTorch. La prueba fija la misma arquitectura y los mismos datos de entrada para comparar el forward pass, el error calculado y la actualización de los parámetros. Al ejecutar los mismos ejemplos, las salidas de la red de la aplicación coinciden con las de la red de PyTorch dentro de las diferencias esperables por precisión numérica. Esta comprobación permite verificar que los pesos, los sesgos y las funciones de activación se aplican en el mismo orden que en una implementación estándar.

Con todas estas mejoras, la aplicación queda organizada en torno a un flujo de uso que integra navegación, adaptación visual, explicación matemática, evaluación y persistencia. El menú principal contextualiza los algoritmos, las vistas se adaptan a distintos dispositivos, las fórmulas se renderizan desde el motor también en web, los conjuntos de datos se seleccionan con más control, la evaluación del árbol avanza paso a paso, la red ofrece explicaciones a varias escalas, las funciones de activación se acompañan con gráficas y los modelos pueden cargarse, guardarse o contrastarse con herramientas externas. El proyecto termina así con una correspondencia más precisa y probada entre los cálculos internos, su representación visual y su explicación textual.

Conclusiones

El desarrollo realizado permite concluir que los objetivos planteados al inicio del trabajo se han cumplido. El resultado final es una aplicación educativa interactiva, accesible desde la web, que permite estudiar de forma visual el entrenamiento y la inferencia de un árbol de decisión y de una red neuronal multicapa. Para llegar a ese resultado se partió del estudio conceptual de ambos algoritmos y se trasladaron sus operaciones fundamentales a una implementación propia en Godot, manteniendo una correspondencia directa entre el cálculo interno, la representación gráfica y las explicaciones mostradas al usuario.

El proceso seguido comenzó con la validación de Godot como entorno adecuado para construir una aplicación interactiva exportable a navegador. Esta primera fase permitió comprobar que era posible representar estructuras dinámicas, modificar elementos durante la ejecución y desplegar el proyecto mediante GitHub Pages. A partir de esa base se incorporó el árbol de decisión, pasando de una estructura editable manualmente a una construcción guiada por el algoritmo ID3. La aplicación permite observar cómo se crean los nodos, cómo se seleccionan los atributos mediante la métrica basada en entropía y cómo se forman las hojas. La ventana informativa, las gráficas y la navegación por pasos convierten ese proceso en una explicación progresiva, en la que cada decisión del algoritmo queda asociada a los datos que la provocan.

El segundo bloque principal fue la red neuronal. Su desarrollo exigió representar una arquitectura formada por capas, neuronas y conexiones densas, y vincular esa representación con un modelo lógico que conserva pesos, sesgos, funciones de activación y parámetros de entrenamiento. La aplicación permite configurar la red según el conjunto de datos, ejecutar el forward pass paso a paso, mostrar el cálculo de cada neurona, comparar la salida con el valor esperado y representar el backward pass mediante la actualización visible de los parámetros. De esta forma, el entrenamiento deja de presentarse como una operación opaca y pasa a mostrarse como una secuencia de predicciones, errores y correcciones sobre el modelo.

La fase final del desarrollo consolidó la aplicación como recurso educativo. Se revisó la navegación general, se adaptaron las vistas a distintos tamaños de pantalla, se mejoró la le-

gibilidad visual de los algoritmos, se integró el renderizado de fórmulas matemáticas y se reforzaron las validaciones de datos. También se completaron los modos de evaluación tanto en el árbol, donde los datos recorren las ramas aprendidas hasta llegar a una hoja, como en la red, donde la inferencia reutiliza el forward pass sin modificar los parámetros entrenados. Estas funciones permiten observar no solo cómo se obtiene un modelo, sino también cómo se utiliza después para clasificar o predecir nuevos ejemplos.

El trabajo ha permitido adquirir competencias directamente relacionadas con los objetivos propuestos. La implementación desde cero de los algoritmos obligó a comprender sus parámetros, sus estructuras internas y las condiciones que aparecen durante su ejecución. La construcción de la interfaz exigió organizar Escenas, Señales, paneles, animaciones y elementos informativos para que la interacción fuera comprensible. El despliegue web, junto con la carga y descarga de archivos en navegador, amplió el alcance técnico del proyecto y permitió que la aplicación pudiera consultarse como recurso público de aprendizaje.

En conjunto, el proyecto cumple el propósito principal de ofrecer una herramienta que ayude a comprender paso a paso el funcionamiento interno de dos modelos básicos de aprendizaje automático. La aplicación final no se limita a mostrar el resultado de un entrenamiento, sino que hace visible el proceso que conduce a ese resultado, acompaña los cálculos con texto, fórmulas y gráficas, y permite experimentar con datos y configuraciones desde una interfaz accesible en web.

Trabajo futuro

El desarrollo realizado deja una base sobre la que pueden plantearse varias ampliaciones. La aplicación actual se centra en dos modelos concretos, árboles de decisión y redes neuronales multicapa, y los presenta de forma incremental para que el usuario pueda seguir tanto el entrenamiento como la evaluación. A partir de esa base, el trabajo futuro puede organizarse en tres líneas principales: ampliar el catálogo de algoritmos, profundizar en las opciones de los modelos ya implementados e integrar el aprendizaje con simulaciones interactivas creadas en el propio motor.

8.1 Ampliación del catálogo de modelos

Una primera línea de evolución consiste en incorporar nuevos algoritmos de aprendizaje automático. La estructura modular desarrollada para separar algoritmo, representación visual y ventana informativa facilita que la aplicación pueda crecer con nuevos modelos, siempre que cada uno se acompañe de una visualización adaptada a su forma de aprender.

En el caso de las redes neuronales, una extensión natural sería incluir topologías más variadas. La versión actual trabaja con redes densas multicapa, suficientes para explicar el forward pass, el cálculo del error y la retropropagación. A partir de ahí, podrían añadirse arquitecturas convolucionales, recurrentes o basadas en mecanismos de atención. Cada una de ellas permitiría trabajar conceptos nuevos: filtros y mapas de características en redes convolucionales, dependencias temporales en redes recurrentes o matrices de atención en modelos de tipo transformer.

Los transformadores tendrían un interés especial como línea futura por su presencia en muchos sistemas actuales de aprendizaje automático. Su incorporación requeriría representar elementos distintos a los de una red densa clásica, como tokens, embeddings, cabezas de atención y bloques sucesivos de procesamiento. Esta ampliación exigiría una visualización más compleja, con relaciones simultáneas entre muchos elementos y conexiones que van más

allá de las capas consecutivas de una red densa.

La incorporación de nuevos modelos también permitiría comparar enfoques. La aplicación podría mostrar cómo distintos algoritmos resuelven un mismo conjunto de datos, qué información utiliza cada uno y qué tipo de explicación resulta más adecuada en cada caso. Esta comparación reforzaría el objetivo didáctico del proyecto, ya que ayudaría a entender que el aprendizaje automático agrupa técnicas con estructuras y procesos de decisión muy diferentes.

8.2 Profundización en los modelos existentes

Una segunda línea de trabajo consiste en enriquecer los algoritmos que ya forman parte de la aplicación. En el árbol de decisión, una mejora directa sería permitir distintos criterios de división. La versión actual se apoya en una métrica basada en entropía e información, por lo que el usuario sigue siempre el mismo tipo de razonamiento al crear cada nodo. Incluir alternativas como el índice Gini o el ratio de ganancia permitiría comparar cómo cambia la elección de atributos según la medida utilizada.

También sería relevante ampliar el tipo de datos aceptados por el árbol. En su estado actual, el algoritmo trabaja con atributos categóricos. Para manejar atributos numéricos continuos habría que buscar umbrales de corte y explicar por qué un valor separa mejor los datos que otros posibles valores. Esta mejora enriquecería la visualización, ya que la ventana informativa podría mostrar la comparación entre atributos y, dentro de cada atributo numérico, la comparación entre posibles puntos de división.

Otra ampliación posible para el árbol sería incorporar técnicas de poda o mecanismos de control de crecimiento. Con ello, el usuario podría observar la diferencia entre construir un árbol que se ajusta mucho a los datos de entrenamiento y construir una estructura más sencilla con mejor capacidad de generalización. Esta línea conectaría la visualización del algoritmo con problemas habituales del aprendizaje automático, como el sobreajuste y la validación del modelo.

En la red neuronal, el trabajo futuro puede avanzar hacia una configuración más flexible del entrenamiento. Algunas opciones posibles son permitir distintos optimizadores, definir el número de épocas, ajustar el tamaño de batch, modificar la tasa de aprendizaje durante la ejecución, elegir funciones de pérdida alternativas o incorporar regularización. Cada una de estas decisiones afecta al modo en el que cambian los pesos, por lo que la aplicación podría mostrar sus efectos de forma visual durante el backward pass.

La red también podría dejar de limitarse a conexiones densas. Permitir topologías no densas o definidas por el usuario abriría la puerta a explicar cómo cambia el flujo de información cuando una neurona no está conectada con todas las neuronas de la capa siguiente. Esta me-

jora mantendría la relación entre arquitectura visible y modelo lógico, a la vez que daría más importancia a la estructura de conexiones como parte del aprendizaje.

Todas estas ampliaciones comparten una misma condición: las nuevas opciones deberían integrarse sin perder el carácter explicativo de la herramienta. Añadir parámetros o variantes del algoritmo tendría sentido si la aplicación muestra qué cambia en el cálculo, qué decisión se toma y qué consecuencia tiene sobre el modelo final.

8.3 Integración con simulaciones interactivas

La tercera línea de trabajo es la más vinculada con la elección de Godot como herramienta de desarrollo. Una evolución especialmente interesante sería conectar los algoritmos con una simulación o un videojuego ejecutándose en la propia aplicación. En ese escenario, el entorno interactivo generaría datos durante la ejecución, y el modelo aprendería a partir de lo que sucede en la escena.

Esta integración permitiría pasar de datasets estáticos a datos producidos por una dinámica viva. Por ejemplo, una escena podría generar posiciones, acciones, colisiones, decisiones de un personaje o resultados obtenidos tras cada intento. Esos eventos podrían convertirse en entradas para un modelo, y la respuesta del modelo podría influir de nuevo en el comportamiento de la simulación. El usuario observaría entonces dos procesos conectados: la evolución del entorno y la evolución del algoritmo que aprende a partir de él.

Una línea de este tipo serviría también para explorar problemas cercanos al aprendizaje por refuerzo. Un agente podría recibir información del estado del juego, escoger acciones y obtener recompensas según lo ocurrido. La visualización tendría que mostrar tanto la ejecución del agente como la actualización interna del modelo. Esto ampliaría el proyecto hacia una dimensión más experimental, donde la explicación del aprendizaje estaría ligada a consecuencias visibles dentro de una escena interactiva.

Esta posibilidad refuerza la justificación del uso de Godot. El motor actuaría como soporte para dibujar interfaces, entorno de simulación, generador de datos y medio de interacción. Además, la exportación a HTML5 permitiría mantener el acceso desde navegador incluso en versiones más cercanas a una experiencia de juego educativo.

En conjunto, estas tres líneas de trabajo permitirían hacer crecer la aplicación en amplitud, profundidad y conexión con entornos interactivos. La prioridad debería seguir siendo la misma que en la versión actual: que cada avance del algoritmo pueda observarse, que cada decisión esté acompañada por una explicación y que la visualización ayude a entender el funcionamiento interno del modelo.

A mayores, sería de gran interés probar la aplicación con usuarios reales para detectar qué partes resultan más y menos claras, y qué cambios podrían mejorar su utilidad didáctica. Sería

especialmente interesante contar con personas en distintos estados del aprendizaje, desde usuarios que se acercan por primera vez al aprendizaje automático hasta estudiantes con las bases asentadas, así como con profesores que pudieran valorar su uso como material de apoyo en clases o actividades prácticas.

Glosario

Escena Las Escenas son el tipo de archivo básico con el que se trabaja en Godot. Estas son una estructura jerárquica de **Nodos**, que parten de un único Nodo raíz. No confundir con *escena* . [iii](#), [3](#), [8–11](#), [15](#), [22](#), [23](#), [25–27](#), [30](#), [33](#), [35–40](#), [42](#), [44](#), [47](#), [51](#), [53](#), [55](#), [65](#), [77](#)

escena Espacio visual en la pantalla. Sinónimo de “vista”. No confundir con *Escena*. [iii](#), [16](#), [22](#), [24](#), [27](#), [28](#), [77](#)

Nodo Los Nodos son la estructura lógica mínima con la que se puede trabajar en Godot. Todo elemento que vaya a estar presente en tiempo de ejecución en el programa es un Nodo. Funcionan como las clases en la programación orientada a objetos, puesto que cada tipo de Nodo cuenta con atributos, funciones y señales propias, además de heredar elementos de su clase padre. No confundir con *nodo*. [3](#), [8–10](#), [22](#), [25](#), [37](#), [38](#), [40](#), [42](#), [63](#), [65](#), [77](#)

nodo Punto de origen de una ramificación. En el contexto de árboles de decisión: representación de una bifurcación producto de una decisión del árbol basada en un atributo de los datos de entrenamiento. En el contexto de redes neuronales: representación del producto escalar entre el vector de salida de los nodos previos y el vector de pesos propio más un valor de sesgo; sinónimo de “neurona”. No confundir con *Nodo*. [iii](#), [v](#), [15](#), [22](#), [23](#), [25–32](#), [34–37](#), [56](#), [59](#), [61](#), [63–65](#), [67](#), [68](#), [77](#)

Bibliografía

- [1] G. Sanderson, “Neural networks,” 3Blue1Brown, consultado el 23 de junio de 2026. [En línea]. Disponible en: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- [2] —, “But what is a neural network?” 3Blue1Brown, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://www.3blue1brown.com/?topic=neural-networks>
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] D. Smilkov and S. Carter, “A neural network playground,” TensorFlow, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://playground.tensorflow.org/>
- [5] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, “Gan lab: Play with generative adversarial networks in your browser!” Georgia Tech and Google Brain/PAIR, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://poloclub.github.io/ganlab/>
- [6] Interactive ML, “Interactive decision trees learning tool,” consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://www.interactive-ml.com/decision-tree.html>
- [7] Godot Engine contributors, “Godot engine,” consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://godotengine.org/>
- [8] —, “Godot documentation,” consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://docs.godotengine.org/>
- [9] —, “Exporting for the web,” consultado el 23 de junio de 2026. [En línea]. Disponible en: https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_web.html
- [10] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>

- [11] GitHub, Inc., “Github pages documentation,” consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://docs.github.com/en/pages>
- [12] K. Schwaber and J. Sutherland, “The scrum guide,” 2020, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://scrumguides.org/scrum-guide.html>
- [13] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [14] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [15] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [16] G. E. Krasner and S. T. Pope, “A cookbook for using the model-view-controller user interface paradigm in smalltalk-80,” *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, 1988, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://www.lri.fr/~mbl/ENS/FONDIHM/2013/papers/Krasner-JOOP88.pdf>
- [17] uetchy, “Math api,” Vercel, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://math.vercel.app/>
- [18] Balsamiq Studios, LLC, “Balsamiq wireframes,” consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://balsamiq.com/wireframes/>
- [19] ratex contributors, “ratex,” crates.io, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://crates.io/crates/ratex>
- [20] R. A. Fisher, “Iris,” UCI Machine Learning Repository, 1936, consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://doi.org/10.24432/C56C76>
- [21] ONNX contributors, “Open Neural Network Exchange,” consultado el 23 de junio de 2026. [En línea]. Disponible en: <https://onnx.ai/>
- [22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. VanderPlas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative

style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.